

EXP NO: 1
DATE:

CREATION OF LARGE DATASET

AIM: To create large datasets and store it in different file formats such as .csv, .json, .xml.

QUESTIONS:

1. Create a 100-record employee dataset of a hospital with the following attributes and save it as csv file (Emp_id, Name, Age, Salary, Designation, Gender, Experience, City and Year of Joining).

CODE:

```
import pandas as pd
import numpy as np
from faker import Faker
import random

fake = Faker()

num_rows = 100

titles = ["Dr.", "Mr.", "Ms."]
designations = [
    "Doctor",
    "Nurse",
    "Radiologist",
    "Pharmacist",
    "Manager",
    "Lab Technician",
    "Paramedic",
    "System Administrator"
]

cities = ["Chennai", "Bangalore", "Hyderabad", "Pune"]

genders = ["Male", "Female"]

data = {
    "Emp_ID": np.arange(1, num_rows + 1),

    "Name": [
        random.choice(titles) + " " + fake.first_name() + " " + fake.last_name()
        for _ in range(num_rows)
    ],

    "Age": np.random.randint(25, 66, num_rows),

    "Salary": np.random.randint(30000, 1000001, num_rows),
```

```

"Designation": np.random.choice(designations, num_rows),

"Gender": np.random.choice(genders, num_rows),

"Experience": np.random.randint(1, 26, num_rows),

"City": np.random.choice(cities, num_rows),

"Year_of_Joining": np.random.randint(2000, 2027, num_rows)
}

df = pd.DataFrame(data)

print(df.head(10))

df.to_csv("Hospital_Employee_Dataset.csv", index=False)

print("CSV File Saved Successfully!")

```

OUTPUT:

```

...   Emp_ID   Name   Age   Salary   Designation   Gender \
0      1  Ms. Matthew Gutierrez   50   42404      Doctor   Female
1      2   Dr. Mary Bennett   62  710563      Doctor   Female
2      3   Ms. Roy Odom   43  920218  Lab Technician   Female
3      4   Dr. James Bell   30  136747      Manager   Female
4      5   Ms. Thomas Hall   54  212726   Radiologist   Female
5      6   Ms. Jose Anderson   42  709740      Nurse     Male
6      7   Mr. John Pratt   25  234730   Radiologist   Female
7      8   Dr. Audrey Castillo   30  476610      Nurse     Male
8      9   Mr. Andrea Holt   52  962177      Doctor   Female
9     10  Ms. Kenneth Harrison   50  773144      Doctor   Female

   Experience   City   Year_of_Joining
0          11  Chennai          2017
1          12  Chennai          2011
2          25  Bangalore          2003
3          11    Pune          2000
4          13  Bangalore          2022
5          18  Chennai          2014
6           8  Bangalore          2008
7           9  Bangalore          2004
8          13  Hyderabad          2006
9           5  Chennai          2009
CSV File Saved Successfully!

```

Hospital_Employee_Dataset

Emp_ID	Name	Age	Salary	Designation	Gender	Experience	City	Year_of_Joining
1	Ms. Matthew Gutierrez	50	42404	Doctor	Female	11	Chennai	2017
2	Dr. Mary Bennett	62	710563	Doctor	Female	12	Chennai	2011
3	Ms. Roy Odom	43	920218	Lab Technician	Female	25	Bangalore	2003
4	Dr. James Bell	30	136747	Manager	Female	11	Pune	2000
5	Ms. Thomas Hall	54	212726	Radiologist	Female	13	Bangalore	2022
6	Ms. Jose Anderson	42	709740	Nurse	Male	18	Chennai	2014
7	Mr. John Pratt	25	234730	Radiologist	Female	8	Bangalore	2008
8	Dr. Audrey Castillo	30	476610	Nurse	Male	9	Bangalore	2004
9	Mr. Andrea Holt	52	962177	Doctor	Female	13	Hyderabad	2006
10	Ms. Kenneth Harrison	50	773144	Doctor	Female	5	Chennai	2009
11	Dr. Misty Pierce	29	481378	Doctor	Male	15	Bangalore	2019
12	Dr. Rachel Perry	59	231853	Manager	Female	4	Hyderabad	2011
13	Mr. Maria McBride	51	352674	System Administrator	Female	14	Chennai	2008
14	Ms. David Peterson	46	234515	Lab Technician	Female	12	Pune	2024
15	Dr. Lauren Morgan	56	181042	Pharmacist	Male	22	Chennai	2012

2. Create a 100-record news article dataset with the following attributes and save it as json file (article_id, title, author, publisher, category, published_date, content, views, likes, comments, shares, reading_time_minutes).

CODE:

```
import pandas as pd
import numpy as np
from faker import Faker
import random
```

```
fake = Faker()
```

```
num_rows = 100
```

```
publishers = [
    "Daily News",
    "Global Times",
    "Tech Today",
    "Sports Weekly",
    "Health Chronicle",
    "Business World"
]
```

```
categories = [
    "Politics",
    "Technology",
    "Sports",
    "Health",
    "Business",
]
```

```

"Entertainment",
"Education",
"Science"
]

data = {
    "article_id": [f"ART{str(i).zfill(3)}" for i in range(1, num_rows + 1)],
    "title": [fake.sentence(nb_words=6) for _ in range(num_rows)],
    "author": [fake.name() for _ in range(num_rows)],
    "publisher": np.random.choice(publishers, num_rows),
    "category": np.random.choice(categories, num_rows),
    "published_date": [
        fake.date_between(start_date='-2y', end_date='today').isoformat()
        for _ in range(num_rows)
    ],
    "content": [fake.text(max_nb_chars=300) for _ in range(num_rows)],
    "views": np.random.randint(100, 100001, num_rows),
    "likes": np.random.randint(10, 10001, num_rows),
    "comments": np.random.randint(0, 2001, num_rows),
    "shares": np.random.randint(0, 5001, num_rows),
    "reading_time_minutes": np.random.randint(2, 16, num_rows)
}

df = pd.DataFrame(data)

print(df.head())

df.to_json("News_Articles.json", orient="records", lines=True)

print("JSON File Saved Successfully!")

```

OUTPUT:

```

...  article_id                                title \
0    ART001      Trade recognize beautiful how matter any.
1    ART002      South month him car follow.
2    ART003  Out own computer camera rise member there hist...
3    ART004      Firm product box specific east.
4    ART005      Toward blood PM ten born.

...  author      publisher      category  published_date \
0    Willie Smith    Daily News      Science    2025-10-03
1    Breanna Mcguire  Sports Weekly    Health    2026-03-14
2    Summer Williams  Business World    Technology  2025-03-04
3    Luis Day        Global Times    Education  2026-03-24
4    Kelly Lewis     Business World  Entertainment  2026-02-04

...  content      views  likes  comments \
0    Black important test. Page century situation w...  73442  5012  336
1    Open shake name detail agreement radio informa...  4986  6807  1188
2    Statement program data above woman somebody ne...  76975  6056  1224
3    Activity difficult young financial computer us...  72287  4625  1834
4    Identify camera work still then. Knowledge dea...  76990  865  1674

...  shares  reading_time_minutes
0    3040      10
1    4990      12
2    20      9
3    461      3
4    1415     13
JSON File Saved Successfully!

```

```

{} News_Articles.json 1 X
Users > john > Desktop > {} News_Articles.json > ...
1 {"article_id":"ART001","title":"Trade recognize beautiful how matter any.,"author":"Willie Smith"
2 {"article_id":"ART002","title":"South month him car follow.,"author":"Breanna Mcguire","publisher
3 {"article_id":"ART003","title":"Out own computer camera rise member there history.,"author":"Summ
4 {"article_id":"ART004","title":"Firm product box specific east.,"author":"Luis Day","publisher":"
5 {"article_id":"ART005","title":"Toward blood PM ten born.,"author":"Kelly Lewis","publisher":"Bus
6 {"article_id":"ART006","title":"Especially lay later case eight.,"author":"Karen Caldwell","publi
7 {"article_id":"ART007","title":"Off safe since meeting ability.,"author":"Samantha Espinoza","pub
8 {"article_id":"ART008","title":"Dog finally family second catch teach.,"author":"Stephanie Maynar
9 {"article_id":"ART009","title":"Animal performance social.,"author":"Jenny Cole","publisher":"Glo
10 {"article_id":"ART010","title":"Certain list down seat level situation.,"author":"Lisa Miller","p
11 {"article_id":"ART011","title":"Hold little east usually behavior piece process produce.,"author"
12 {"article_id":"ART012","title":"Night economy ok prepare true believe election.,"author":"Carolín
13 {"article_id":"ART013","title":"Attention least modern election total theory hard.,"author":"Mich
14 {"article_id":"ART014","title":"Evidence rest friend nice establish marriage rate street.,"author
15 {"article_id":"ART015","title":"Coach despite drop environmental check.,"author":"Sarah Franklin"

```

3. Create a 100-record Purchase dataset with the following attributes and save it as xml file (DeviceID, OwnerName, IPAddress, OS, OSVersion, PurchaseDate, Warranty Status, BatteryHealth, IsOnline).

CODE:

```

import pandas as pd
from faker import Faker
import random

fake = Faker()

num_rows = 100

os_list = ["Windows", "Linux", "Android", "iOS", "macOS"]
warranty = ["Valid", "Expired"]

data = {
    "DeviceID": [fake.uuid4() for _ in range(num_rows)],
    "OwnerName": [fake.name() for _ in range(num_rows)],
    "IPAddress": [fake.ipv4() for _ in range(num_rows)],
    "OS": [random.choice(os_list) for _ in range(num_rows)],
    "OSVersion": [fake.numerify(text="###") for _ in range(num_rows)],
    "PurchaseDate": [
        fake.date_between(start_date='-5y', end_date='today').isoformat()
        for _ in range(num_rows)
    ],
    "WarrantyStatus": [random.choice(warranty) for _ in range(num_rows)],
    "BatteryHealth": [random.randint(50, 100) for _ in range(num_rows)],
    "IsOnline": [random.choice([True, False]) for _ in range(num_rows)]
}

df = pd.DataFrame(data)

print(df.head())

```

```
df.to_xml(
    "Purchase_Dataset.xml",
    root_name="Devices",
    row_name="Device",
    index=False
)

print("XML File Saved Successfully!")
```

OUTPUT:

```
...
      DeviceID      OwnerName      IPAddress \
0  56609355-2510-4ac0-94ca-27fbc4079dd1      Tina West      221.212.19.52
1  ef1e7691-3425-484e-b713-7fe629437ad7      Nancy Fleming      28.125.221.95
2  69fc31b2-4ffa-4dc4-9740-b9b19115f8a6      Eric Spence      82.250.169.129
3  32217122-5308-4829-9e7b-8c44f3bb485b      Mr. Michael Leach      1.172.235.215
4  b4571dac-8678-4eae-ba7c-440f27d95a4d      Carolyn Hill      202.93.75.124

      OS OSVersion PurchaseDate WarrantyStatus BatteryHealth IsOnline
0   Linux      9.8.7      2024-06-10           Valid           83      False
1    iOS      0.0.4      2024-10-06          Expired           95      False
2  Windows      6.8.0      2026-04-19          Expired           66      False
3    iOS      6.0.0      2024-11-12           Valid           51      False
4    iOS      7.2.0      2025-01-02           Valid           72       True
XML File Saved Successfully!
```

```
Purchase_Dataset.xml X
Users > john > Desktop > Purchase_Dataset.xml
1  <?xml version='1.0' encoding='utf-8'?>
2  <Devices>
3    <Device>
4      <DeviceID>56609355-2510-4ac0-94ca-27fbc4079dd1</DeviceID>
5      <OwnerName>Tina West</OwnerName>
6      <IPAddress>221.212.19.52</IPAddress>
7      <OS>Linux</OS>
8      <OSVersion>9.8.7</OSVersion>
9      <PurchaseDate>2024-06-10</PurchaseDate>
10     <WarrantyStatus>Valid</WarrantyStatus>
11     <BatteryHealth>83</BatteryHealth>
12     <IsOnline>False</IsOnline>
13   </Device>
14   <Device>
15     <DeviceID>ef1e7691-3425-484e-b713-7fe629437ad7</DeviceID>
```

RESULT: Hence, large datasets were successfully created and exported using Python libraries.

EXP NO: 2**DATA PREPROCESSING****DATE:**

AIM: To perform various data preprocessing techniques

QUESTIONS:**1. Import and Explore Dataset****1.1 Import the required libraries.****1.2 Load the dataset into a Pandas DataFrame.****1.3 Display the first 5 and last 5 records of the dataset.****1.4 Determine the number of rows and columns in the dataset.****1.5 Display the data types of all attributes.****1.6 Generate summary statistics for the numerical features.****CODE:**

1.1 Import Libraries

import pandas as pd

import numpy as np

from sklearn.preprocessing import LabelEncoder

from sklearn.preprocessing import MinMaxScaler

from sklearn.preprocessing import StandardScaler

import matplotlib.pyplot as plt

1.2 Load Dataset

df = pd.read_csv("Bank_Marketing_Synthetic_500.csv")

1.3 Display First and Last 5 Records

print("First 5 Records")

print(df.head())

print("\nLast 5 Records")

print(df.tail())

1.4 Number of Rows and Columns

print("\nDataset Shape")

print(df.shape)

print("Rows:", df.shape[0])

print("Columns:", df.shape[1])

1.5 Display Data Types

print("\nData Types")

print(df.dtypes)


```
# 1.6 Summary Statistics
print("\nSummary Statistics")
print(df.describe())
```

OUTPUT:

```
First 5 Records
   age  job  marital  education  default  balance  housing  loan  contact \
0   56  retired  divorced  unknown    no  33262.0    no  yes  unknown
1   69  unknown    single  secondary    no  10677.0    yes  no   unknown
2   46  unknown  married  secondary    no  49871.0    yes  no  telephone
3   32  unknown  married  secondary    no  16268.0    no  no   telephone
4   60  retired  married   unknown    no  10330.0    yes  no   cellular

   day month  duration  campaign  pdays  previous  poutcome  y
0   26  dec   1387.0     11    244         5    other    no
1    4   may   145.0     11    310         5  unknown    no
2   10   mar  2125.0     11     58         8  failure    no
3    5   dec  1831.0     14    277         3  unknown    no
4   21   mar  2176.0      9    101         0  unknown    no

Last 5 Records
   age  job  marital  education  default  balance  housing  loan  \
505  47  services  divorced  primary    no  44026.0    yes  yes
506  29  blue-collar  divorced  primary    no  48019.0    no   no
507  52  retired  divorced  primary    no  39674.0    no   no
508  31  blue-collar  married  tertiary  yes  5758.0    yes  no
509  57  housemaid   single  unknown    no  40821.0    no   no

   contact  day month  duration  campaign  pdays  previous  poutcome  y
505  unknown  23  aug   1501.0         1    326         2  unknown    no
506  unknown  28  apr   166.0         9    403         1    other    no
507  telephone 27  sep   307.0         7    313         9  success    no
508  cellular   4  may   659.0        13    465         1  unknown  yes
509  unknown  27  nov   302.0         9    339         5  failure    no

Dataset Shape
(510, 17)
Rows: 510
Columns: 17

Data Types
age          int64
job          object
marital      object
education    object
default      object
balance      float64
housing      object
loan         object
contact      object
day          int64
month        object
duration     float64
campaign     int64
pdays       int64
previous     int64
poutcome     object
y           object
dtype: object

Summary Statistics
   count  age  balance  day  duration  campaign \
count  510.000000  505.000000  510.000000  505.000000  510.000000
mean    46.129412  24796.196040  16.945098  1438.736634  7.747059
std     16.162331  14632.699203   8.759666   840.784070  3.945695
min     18.000000  -1839.000000   1.000000   37.000000  1.000000
25%     33.000000  12289.000000  10.000000   682.000000  4.000000
50%     46.000000  25605.000000  17.000000  1439.000000  8.000000
75%     59.000000  36787.000000  24.000000  2131.000000  11.000000
max     74.000000  49871.000000  31.000000  2995.000000  14.000000

   pdays  previous
count  510.000000  510.000000
mean    248.596078   4.741176
std     145.958298   2.963377
min     -1.000000   0.000000
25%     113.250000   2.000000
50%     245.500000   5.000000
75%     376.500000   7.000000
max     498.000000   9.000000
```


2. Identify and Handle Missing Values

2.1 Identify missing values using Pandas functions.

2.2 Treat the value "unknown" as a missing value.

2.3 Calculate the percentage of missing values in each column.

2.4 Handle the missing values.

2.5 Verify that all missing values have been addressed.

CODE:

```
# 2.1 Identify Missing Values
```

```
print("\nMissing Values")
```

```
print(df.isnull().sum())
```

```
# 2.2 Treat "unknown" as Missing Value
```

```
df.replace("unknown", np.nan, inplace=True)
```

```
print("\nMissing Values After Replacing Unknown")
```

```
print(df.isnull().sum())
```

```
# 2.3 Percentage of Missing Values
```

```
missing_percent = (df.isnull().sum()/len(df))*100
```

```
print("\nPercentage of Missing Values")
```

```
print(missing_percent)
```

```
# 2.4 Handle Missing Values
```

```
# Numerical Columns
```

```
num_cols = df.select_dtypes(include=np.number).columns
```

```
for col in num_cols:
```

```
    df[col].fillna(df[col].median(), inplace=True)
```

```
# Categorical Columns
```

```
cat_cols = df.select_dtypes(include='object').columns
```

```
for col in cat_cols:
```

```
    df[col].fillna(df[col].mode()[0], inplace=True)
```

```
# 2.5 Verify Missing Values
```

```
print("\nMissing Values After Handling")
```

```
print(df.isnull().sum())
```

OUTPUT:

Missing Values		Missing Values After Replacing Unknown	
age	0	age	0
job	0	job	62
marital	0	marital	0
education	0	education	128
default	0	default	0
balance	5	balance	5
housing	0	housing	0
loan	0	loan	0
contact	0	contact	178
day	0	day	0
month	0	month	0
duration	5	duration	5
campaign	0	campaign	0
pdays	0	pdays	0
previous	0	previous	0
poutcome	0	poutcome	138
y	0	y	0
dtype: int64		dtype: int64	

Percentage of Missing Values		Missing Values After Handling	
age	0.000000	age	0
job	12.156863	job	0
marital	0.000000	marital	0
education	25.098039	education	0
default	0.000000	default	0
balance	0.980392	balance	0
housing	0.000000	housing	0
loan	0.000000	loan	0
contact	34.901961	contact	0
day	0.000000	day	0
month	0.000000	month	0
duration	0.980392	duration	0
campaign	0.000000	campaign	0
pdays	0.000000	pdays	0
previous	0.000000	previous	0
poutcome	27.058824	poutcome	0
y	0.000000	y	0
dtype: float64		dtype: int64	

3. Remove Duplicate Records

3.1 Find duplicate records.

3.2 Count the number of duplicate records.

3.3 Remove duplicate records.

3.4 Compare the dataset dimensions before and after duplicate removal.

CODE:

```
before = df.shape
```

```
# 3.1 Find Duplicate Records
```

```
duplicates = df[df.duplicated()]
```

```
print("\nDuplicate Records")  
print(duplicates)
```

```
# 3.2 Count Duplicate Records
```

```
print("\nNumber of Duplicate Records")  
print(df.duplicated().sum())
```

```
# 3.3 Remove Duplicate Records
```

```
df = df.drop_duplicates()
```

```
# 3.4 Compare Dataset Dimensions
```

```
after = df.shape
```

```
print("\nBefore Removing Duplicates:", before)  
print("After Removing Duplicates :", after)
```

OUTPUT:

```

Duplicate Records
  age  job  marital  education  default  balance  housing  loan  \
500  58  retired  divorced  primary    no  14029.0    yes   no
501  21  housemaid  married  tertiary   no   3542.0    yes   no
502  34  housemaid  divorced  secondary  no  35861.0    no   no
503  22   admin.  married  primary    no   8142.0    no   no
504  42  unemployed  divorced  secondary  no  28620.0    no   no
506  29  blue-collar  divorced  primary    no  48019.0    no   no
507  52   retired  divorced  primary    no  39674.0    no   no
508  31  blue-collar  married  tertiary   yes   5758.0    yes   no
509  57  housemaid   single  primary    no  40821.0    no   no

  contact  day month  duration  campaign  pdays  previous  poutcome  y
500  telephone    8  feb   1962.0      7     69         7    other    no
501  telephone   20  apr   1835.0      2    230         9  failure  yes
502   cellular   28  mar   1079.0      9    368         3    other  yes
503   cellular   24  nov   2393.0      3     85         5  failure  no
504  telephone   29  mar   2298.0      6    329         7  failure  yes
506   cellular   28  apr    166.0      9    403         1    other    no
507  telephone   27  sep    307.0      7    313         9  success  no
508   cellular    4  may    659.0     13    465         1    other  yes
509   cellular   27  nov    302.0      9    339         5  failure  no

Number of Duplicate Records
9

Before Removing Duplicates: (510, 17)
After Removing Duplicates : (501, 17)

```

4. Perform Categorical Data Encoding**4.1 Identify the categorical attributes.****4.2 Apply Label Encoding to binary categorical variables.****4.3 Apply One-Hot Encoding to multi-categorical variables.****4.4 Compare the dataset before and after encoding.****CODE:**

```
# 4.1 Identify Categorical Attributes
```

```
categorical = df.select_dtypes(include='object').columns
```

```
print("\nCategorical Columns")
```

```
print(categorical)
```

```
encoded_df = df.copy()
```

```
# 4.2 Label Encoding for Binary Variables
```

```
le = LabelEncoder()
```

```
for col in categorical:
```

```
if encoded_df[col].nunique()==2:  
    encoded_df[col] = le.fit_transform(encoded_df[col])
```

4.3 One-Hot Encoding

```
multi_cols = []
```

```
for col in categorical:
```

```
    if df[col].nunique()>2:  
        multi_cols.append(col)
```

```
encoded_df = pd.get_dummies(encoded_df,  
                             columns=multi_cols,  
                             drop_first=True)
```

4.4 Compare Dataset

```
print("\nOriginal Dataset")  
print(df.head())
```

```
print("\nEncoded Dataset")  
print(encoded_df.head())
```

OUTPUT:

```
Categorical Columns
Index(['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact',
      'month', 'poutcome', 'y'],
      dtype='object')
```

Original Dataset

	age	job	marital	education	default	balance	housing	loan	\
0	56	retired	divorced	primary	no	33262.0	no	yes	
1	69	housemaid	single	secondary	no	10677.0	yes	no	
2	46	housemaid	married	secondary	no	49871.0	yes	no	
3	32	housemaid	married	secondary	no	16268.0	no	no	
4	60	retired	married	primary	no	10330.0	yes	no	

	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	cellular	26	dec	1387.0	11	244	5	other	no
1	cellular	4	may	145.0	11	310	5	other	no
2	telephone	10	mar	2125.0	11	58	8	failure	no
3	telephone	5	dec	1831.0	14	277	3	other	no
4	cellular	21	mar	2176.0	9	101	0	other	no

Encoded Dataset

	age	default	balance	housing	loan	contact	day	duration	campaign	\
0	56	0	33262.0	0	1	0	26	1387.0	11	
1	69	0	10677.0	1	0	0	4	145.0	11	
2	46	0	49871.0	1	0	1	10	2125.0	11	
3	32	0	16268.0	0	0	1	5	1831.0	14	
4	60	0	10330.0	1	0	0	21	2176.0	9	

	pdays	...	month_jan	month_jul	month_jun	month_mar	month_may	\
0	244	...	False	False	False	False	False	
1	310	...	False	False	False	False	True	
2	58	...	False	False	False	True	False	
3	277	...	False	False	False	False	False	
4	101	...	False	False	False	True	False	

	month_nov	month_oct	month_sep	poutcome_other	poutcome_success
0	False	False	False	True	False
1	False	False	False	True	False
2	False	False	False	False	False
3	False	False	False	True	False
4	False	False	False	True	False

```
[5 rows x 39 columns]
```

5. Normalize and Standardize Numerical Values**5.1 Identify the numerical features.****5.2 Apply Min-Max Normalization.****5.3 Apply Z-score Standardization.****5.4 Compare the transformed values with the original dataset.**

CODE:

5.1 Numerical Features

```
numerical = encoded_df.select_dtypes(include=np.number).columns
```

```
print("\nNumerical Features")  
print(numerical)
```

5.2 Min-Max Normalization

```
minmax = MinMaxScaler()
```

```
normalized = encoded_df.copy()
```

```
normalized[numerical] = minmax.fit_transform(normalized[numerical])
```

```
print("\nMin-Max Normalized Data")  
print(normalized.head())
```

5.3 Standardization (Z-score)

```
standard = StandardScaler()
```

```
standardized = encoded_df.copy()
```

```
standardized[numerical] = standard.fit_transform(standardized[numerical])
```

```
print("\nStandardized Data")  
print(standardized.head())
```

5.4 Compare

```
print("\nOriginal Values")  
print(encoded_df[numerical].head())
```

```
print("\nNormalized Values")  
print(normalized[numerical].head())
```

```
print("\nStandardized Values")  
print(standardized[numerical].head())
```


OUTPUT:

```

Numerical Features
Index(['age', 'default', 'balance', 'housing', 'loan', 'contact', 'day',
      'duration', 'campaign', 'pdays', 'previous', 'y'],
      dtype='object')

Min-Max Normalized Data
   age  default  balance  housing  loan  contact  day  duration \
0  0.678571    0.0  0.678805    0.0    1.0    0.0  0.833333  0.456389
1  0.910714    0.0  0.242042    1.0    0.0    0.0  0.100000  0.036511
2  0.500000    0.0  1.000000    1.0    0.0    1.0  0.300000  0.705882
3  0.250000    0.0  0.350164    0.0    0.0    1.0  0.133333  0.606491
4  0.750000    0.0  0.235332    1.0    0.0    0.0  0.666667  0.723124

   campaign  pdays  ...  month_jan  month_jul  month_jun  month_mar \
0  0.769231  0.490982  ...    False    False    False    False
1  0.769231  0.623246  ...    False    False    False    False
2  0.769231  0.118236  ...    False    False    False    True
3  1.000000  0.557114  ...    False    False    False    False
4  0.615385  0.204409  ...    False    False    False    True

   month_may  month_nov  month_oct  month_sep  poutcome_other \
0    False    False    False    False    True
1    True    False    False    False    True
2    False    False    False    False    False
3    False    False    False    False    True
4    False    False    False    False    True

   poutcome_success
0    False
1    False
2    False
3    False
4    False

[5 rows x 39 columns]

Standardized Data
   age  default  balance  housing  loan  contact  day \
0  0.602435 -0.224309  0.582861 -1.134756  2.053960 -0.684988  1.047323
1  1.407124 -0.224309 -0.973098  0.881247 -0.486864 -0.684988 -1.473662
2 -0.016556 -0.224309  1.727113  0.881247 -0.486864  1.459880 -0.786121
3 -0.883144 -0.224309 -0.587914 -1.134756 -0.486864  1.459880 -1.359072
4  0.850032 -0.224309 -0.997004  0.881247 -0.486864 -0.684988  0.474372

   duration  campaign  pdays  ...  month_jan  month_jul  month_jun \
0 -0.066623  0.820333 -0.026507  ...    False    False    False
1 -1.554135  0.820333  0.425526  ...    False    False    False
2  0.817261  0.820333 -1.300421  ...    False    False    False
3  0.465145  1.579078  0.199510  ...    False    False    False
4  0.878343  0.314503 -1.005914  ...    False    False    False

```

2	True	False	False	False	False	False
3	False	False	False	False	False	True
4	True	False	False	False	False	True

	poutcome_success
0	False
1	False
2	False
3	False
4	False

[5 rows x 39 columns]

Original Values

	age	default	balance	housing	loan	contact	day	duration	campaign	\
0	56	0	33262.0	0	1	0	26	1387.0	11	
1	69	0	10677.0	1	0	0	4	145.0	11	
2	46	0	49871.0	1	0	1	10	2125.0	11	
3	32	0	16268.0	0	0	1	5	1831.0	14	
4	60	0	10330.0	1	0	0	21	2176.0	9	

	pdays	previous	y
0	244	5	0
1	310	5	0
2	58	8	0
3	277	3	0
4	101	0	0

Normalized Values

	age	default	balance	housing	loan	contact	day	duration	\
0	0.678571	0.0	0.678805	0.0	1.0	0.0	0.833333	0.456389	
1	0.910714	0.0	0.242042	1.0	0.0	0.0	0.100000	0.036511	
2	0.500000	0.0	1.000000	1.0	0.0	1.0	0.300000	0.705882	
3	0.250000	0.0	0.350164	0.0	0.0	1.0	0.133333	0.606491	
4	0.750000	0.0	0.235332	1.0	0.0	0.0	0.666667	0.723124	

	campaign	pdays	previous	y
0	0.769231	0.490982	0.555556	0.0
1	0.769231	0.623246	0.555556	0.0
2	0.769231	0.118236	0.888889	0.0
3	1.000000	0.557114	0.333333	0.0
4	0.615385	0.204409	0.000000	0.0

Standardized Values

	age	default	balance	housing	loan	contact	day	\
0	0.602435	-0.224309	0.582861	-1.134756	2.053960	-0.684988	1.047323	
1	1.407124	-0.224309	-0.973098	0.881247	-0.486864	-0.684988	-1.473662	
2	-0.016556	-0.224309	1.727113	0.881247	-0.486864	1.459880	-0.786121	
3	-0.883144	-0.224309	-0.587914	-1.134756	-0.486864	1.459880	-1.359072	
4	0.850032	-0.224309	-0.997004	0.881247	-0.486864	-0.684988	0.474372	

	duration	campaign	pdays	previous	y
0	-0.066623	0.820333	-0.026507	0.090333	-0.42615
1	-1.554135	0.820333	0.425526	0.090333	-0.42615
2	0.817261	0.820333	-1.300421	1.103547	-0.42615
3	0.465145	1.579078	0.199510	-0.585143	-0.42615
4	0.878343	0.314503	-1.005914	-1.598356	-0.42615

6. Detect and Handle Outliers

6.1 Plot boxplots for the numerical attributes.

6.2 Detect outliers using the IQR method.

6.3 Remove or cap the outliers.

6.4 Compare the dataset before and after outlier treatment.

CODE:

```
# Add some outliers

df.loc[0, "balance"] = 150000
df.loc[1, "balance"] = 200000
df.loc[2, "balance"] = -50000

df.loc[3, "duration"] = 10000
df.loc[4, "duration"] = 12000

df.loc[5, "campaign"] = 50
df.loc[6, "campaign"] = 60

df.loc[7, "age"] = 99
df.loc[8, "age"] = 100

numeric = df.select_dtypes(include=np.number).columns

# 6.1 Boxplots

for col in numeric:

    plt.figure(figsize=(5,2))
    plt.boxplot(df[col], vert=False)
    plt.title(col)
    plt.show()

# 6.2 Detect Outliers using IQR

outlier_count = {}

for col in numeric:

    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3-Q1

    lower = Q1 - 1.5*IQR
    upper = Q3 + 1.5*IQR

    outliers = df[(df[col]<lower)|(df[col]>upper)]
```

```

    outlier_count[col] = len(outliers)

print("\nOutlier Count")

for k,v in outlier_count.items():
    print(k,":",v)

# 6.3 Cap Outliers

treated_df = df.copy()

for col in numeric:

    Q1 = treated_df[col].quantile(0.25)
    Q3 = treated_df[col].quantile(0.75)

    IQR = Q3-Q1

    lower = Q1 - 1.5*IQR
    upper = Q3 + 1.5*IQR

    treated_df[col] = np.where(
        treated_df[col]<lower,
        lower,
        treated_df[col]
    )

    treated_df[col] = np.where(
        treated_df[col]>upper,
        upper,
        treated_df[col]
    )

# 6.4 Compare

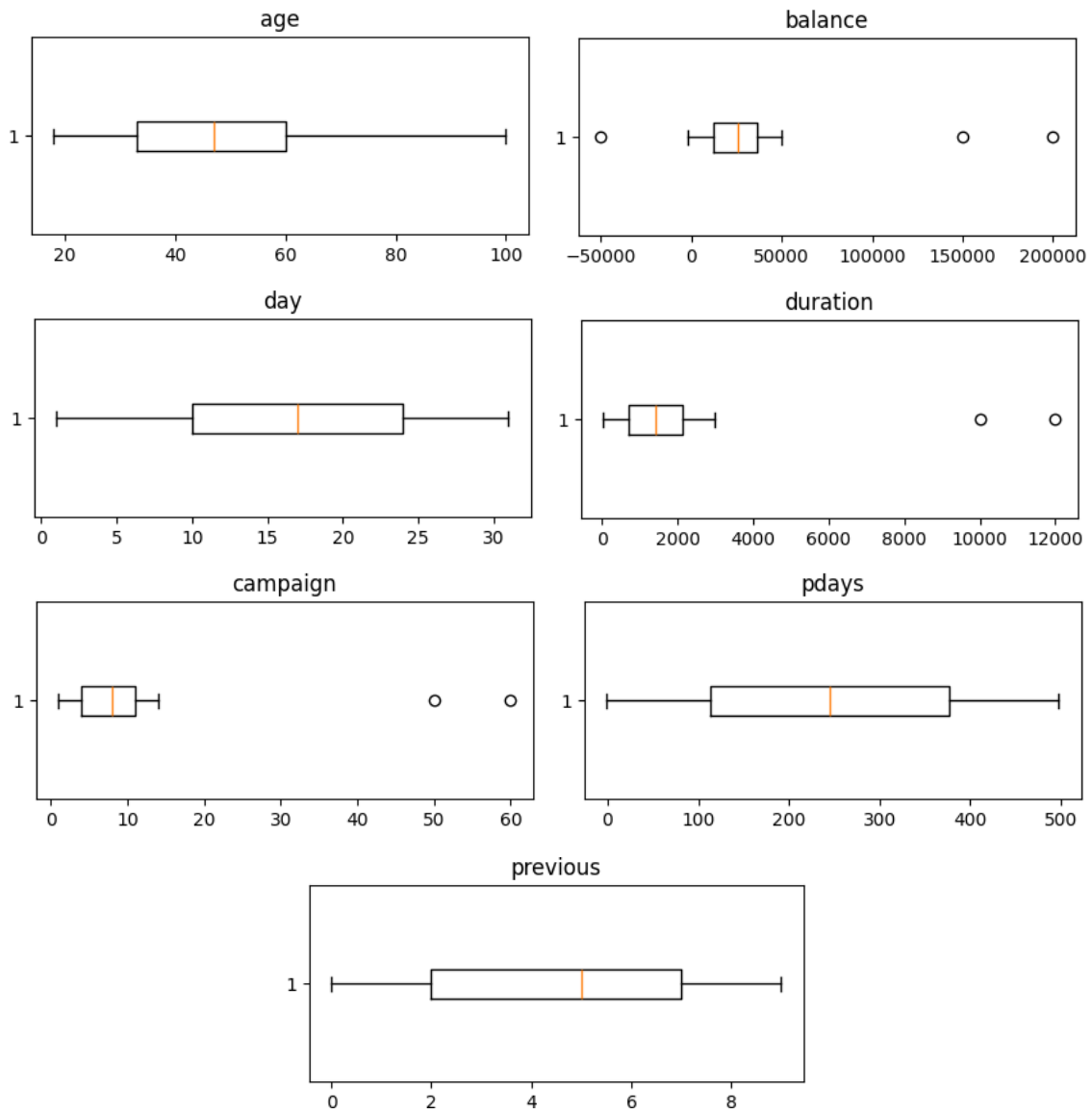
print("\nOriginal Dataset Shape")
print(df.shape)

print("\nAfter Outlier Treatment")
print(treated_df.shape)

print("\nOriginal Data")
print(df.head())

print("\nAfter Outlier Treatment")
print(treated_df.head())

```

OUTPUT:

Outlier Count

```
age : 0
balance : 3
day : 0
duration : 2
campaign : 2
pdays : 0
previous : 0
```

Original Dataset Shape
(501, 17)

After Outlier Treatment
(501, 17)

Original Data

	age	job	marital	education	default	balance	housing	loan	\
0	56	retired	divorced	primary	no	150000.0	no	yes	
1	69	housemaid	single	secondary	no	200000.0	yes	no	
2	46	housemaid	married	secondary	no	-50000.0	yes	no	
3	32	housemaid	married	secondary	no	16268.0	no	no	
4	60	retired	married	primary	no	10330.0	yes	no	

	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	cellular	26	dec	1387.0	11	244	5	other	no
1	cellular	4	may	145.0	11	310	5	other	no
2	telephone	10	mar	2125.0	11	58	8	failure	no
3	telephone	5	dec	10000.0	14	277	3	other	no
4	cellular	21	mar	12000.0	9	101	0	other	no

After Outlier Treatment

	age	job	marital	education	default	balance	housing	loan	\
0	56.0	retired	divorced	primary	no	73077.0	no	yes	
1	69.0	housemaid	single	secondary	no	73077.0	yes	no	
2	46.0	housemaid	married	secondary	no	-23891.0	yes	no	
3	32.0	housemaid	married	secondary	no	16268.0	no	no	
4	60.0	retired	married	primary	no	10330.0	yes	no	

	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	cellular	26.0	dec	1387.0	11.0	244.0	5.0	other	no
1	cellular	4.0	may	145.0	11.0	310.0	5.0	other	no
2	telephone	10.0	mar	2125.0	11.0	58.0	8.0	failure	no
3	telephone	5.0	dec	4274.5	14.0	277.0	3.0	other	no
4	cellular	21.0	mar	4274.5	9.0	101.0	0.0	other	no

7. Perform Feature Engineering

7.1 Create age groups such as Young, Middle-aged, Seniors.

7.2 Categorise account balances into Low, Medium, and High.

7.3 Display the updated dataset.

CODE:

#7.1 Age groups

```
age_bins = [0, 35, 55, np.inf]
```

```
age_labels = ['Young', 'Middle-aged', 'Senior']
```

```
df['AgeGroup'] = pd.cut(df['age'], bins=age_bins, labels=age_labels, right=True)
```

#7.2 Account Balance groups

```
balance_bins = [0, 5000, 15000, np.inf]
```

```
balance_labels = ['Low', 'Medium', 'High']
```

```
df['BalanceCategory'] = pd.cut(df['balance'], bins=balance_bins, labels=balance_labels, right=True)
```

#7.3 Display

```
df
```

OUTPUT:

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y	AgeGroup	BalanceCategory
0	56	retired	divorced	primary	no	150000.0	no	yes	cellular	26	dec	1387.0	11	244	5	other	no	Senior	High
1	69	housemaid	single	secondary	no	200000.0	yes	no	cellular	4	may	145.0	11	310	5	other	no	Senior	High
2	46	housemaid	married	secondary	no	-50000.0	yes	no	telephone	10	mar	2125.0	11	58	8	failure	no	Middle-aged	NaN
3	32	housemaid	married	secondary	no	16268.0	no	no	telephone	5	dec	10000.0	14	277	3	other	no	Young	High
4	60	retired	married	primary	no	10330.0	yes	no	cellular	21	mar	12000.0	9	101	0	other	no	Senior	Medium
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
496	60	student	single	secondary	no	28682.0	yes	no	cellular	6	oct	949.0	9	432	8	other	no	Senior	High
497	54	services	single	tertiary	no	30262.0	no	no	cellular	31	jul	2389.0	4	123	3	success	no	Middle-aged	High
498	50	student	married	tertiary	no	29358.0	no	no	cellular	6	sep	1999.0	7	79	8	other	no	Middle-aged	High
499	25	unemployed	single	primary	no	12289.0	no	no	cellular	24	sep	785.0	2	483	1	success	no	Young	Medium
505	47	services	divorced	primary	no	44026.0	yes	yes	cellular	23	aug	1501.0	1	326	2	other	no	Middle-aged	High

501 rows x 19 columns

8. Perform Feature Selection

8.1 Compute the correlation matrix for numerical variables.

8.2 Visualize correlations using a heatmap.

8.3 Remove highly correlated features, if any.

8.4 Identify the most relevant features for predicting customer subscription.

CODE:

#8.1 Correlation matrix

```
corr_matrix = df.corr(numeric_only=True)
```

```
print("\nCorrelation Matrix:\n", corr_matrix)
```

#8.2 Heatmap visualisation

```
!pip install seaborn
```

```
import seaborn as sns
```

```
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```

#8.3 Removal of high;y correlated features

```
threshold = 0.5
```

```
upper_tri = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
to_drop = [column for column in upper_tri.columns if any(upper_tri[column].abs() >
threshold)]
print("Highly correlated features to drop:", to_drop)
```

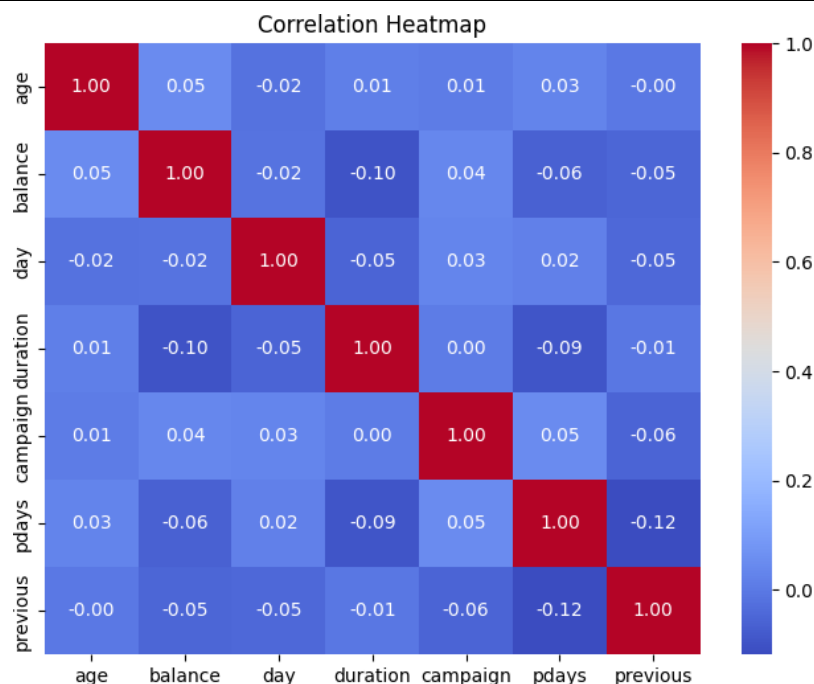
#8.4 Most relevant features for subscription

```
y_numeric = df_reduced['y'].map({'yes': 1, 'no': 0})
numeric_df = df_reduced.select_dtypes(include=['number'])
correlations = numeric_df.apply(lambda col: col.corr(y_numeric)).abs()
relevant_features = correlations.sort_values(ascending=False)
print("\nMost Relevant Numerical Features for Predicting Subscription:")
print(relevant_features)
```

OUTPUT:

Correlation Matrix:

	age	balance	day	duration	campaign	pdays	previous
age	1.000000	0.050481	-0.018648	0.012800	0.005759	0.027218	-0.004520
balance	0.050481	1.000000	-0.016768	-0.098274	0.038116	-0.062032	-0.048985
day	-0.018648	-0.016768	1.000000	-0.053413	0.033793	0.021051	-0.046791
duration	0.012800	-0.098274	-0.053413	1.000000	0.004680	-0.094637	-0.006718
campaign	0.005759	0.038116	0.033793	0.004680	1.000000	0.051114	-0.056918
pdays	0.027218	-0.062032	0.021051	-0.094637	0.051114	1.000000	-0.118763
previous	-0.004520	-0.048985	-0.046791	-0.006718	-0.056918	-0.118763	1.000000



Highly correlated features to drop: []

Most Relevant Numerical Features for Predicting Subscription:

```
previous    0.068048
day         0.047717
pdays      0.041544
campaign    0.025719
balance     0.021007
duration    0.019670
age         0.007371
dtype: float64
```

9. Build a Complete Data Preprocessing Pipeline

9.1 Load the dataset. Handle missing values.

9.2 Remove duplicates.

9.3 Encode categorical variables.

9.4 Normalize numerical attributes.

9.5 Detect and treat outliers.

9.6 Select relevant features.

9.7 Split the dataset into training and testing sets.

9.8 Save the preprocessed dataset as a new CSV file.

CODE:

```
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
def remove_duplicates_and_outliers(df, numeric_cols):
    df = df.drop_duplicates()
    for col in numeric_cols:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]
    return df
```

```
df = pd.read_csv("Bank_Marketing_Synthetic_500_1.csv")
```

```
numeric_features = df.select_dtypes(include=[np.number]).columns.tolist()
categorical_features = df.select_dtypes(exclude=[np.number]).columns.tolist()
```

```

target_column = "y" if "y" in df.columns else df.columns[-1]

if target_column in numeric_features:
    numeric_features.remove(target_column)
if target_column in categorical_features:
    categorical_features.remove(target_column)

df_cleaned = remove_duplicates_and_outliers(df, numeric_features)

X = df_cleaned[numeric_features + categorical_features]
y = df_cleaned[target_column]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

numeric_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="mean")),
        ("scaler", StandardScaler()),
    ]
)

categorical_pipeline = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
    ]
)

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_pipeline, numeric_features),
        ("cat", categorical_pipeline, categorical_features),
    ]
)

k_features = min(5, len(numeric_features) + len(categorical_features))

full_pipeline = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("feature_selection", SelectKBest(score_func=f_classif, k=k_features)),
    ]
)

X_train_processed = full_pipeline.fit_transform(X_train, y_train)
X_test_processed = full_pipeline.transform(X_test)

```

```
feature_names = full_pipeline.named_steps["preprocessor"].get_feature_names_out()
selected_mask = full_pipeline.named_steps["feature_selection"].get_support()
selected_feature_names = feature_names[selected_mask]
```

```
df_train_processed = pd.DataFrame(
    X_train_processed, columns=selected_feature_names
)
df_train_processed[target_column] = y_train.values
```

```
df_test_processed = pd.DataFrame(
    X_test_processed, columns=selected_feature_names
)
df_test_processed[target_column] = y_test.values
```

```
df_train_processed.to_csv("train_preprocessed.csv", index=False)
df_test_processed.to_csv("test_preprocessed.csv", index=False)
```

OUTPUT:

	A	B	C	D	E	F
	test_preprocessed					
	num_campaign	num_previous	cat_job_blue-collar	cat_month_jun	cat_month_sep	y
1						
2	-0.9266922008397330	-0.5431806344167470	0.0	1.0	0.0	no
3	0.5590474422243790	-0.20786282545396800	0.0	0.0	0.0	no
4	0.31142416838036	1.4687262193599300	0.0	0.0	0.0	no
5	-0.679068926995714	0.4627727924715910	0.0	0.0	0.0	no
6	-0.4314456531516960	0.7980906014343700	0.0	1.0	0.0	no
7	0.31142416838036	0.7980906014343700	0.0	0.0	0.0	yes
8	0.5590474422243790	-0.878498443379526	0.0	0.0	0.0	no
9	0.31142416838036	0.4627727924715910	1.0	0.0	0.0	no
10	1.0542939899124200	0.4627727924715910	0.0	0.0	0.0	no
11	-0.679068926995714	1.1334084103971500	0.0	0.0	0.0	no
12	-1.174315474683750	-0.878498443379526	0.0	0.0	0.0	no
13	0.806670716068397	1.4687262193599300	0.0	0.0	0.0	no
14	0.31142416838036	1.4687262193599300	0.0	1.0	0.0	no
15	1.0542939899124200	-0.5431806344167470	0.0	0.0	0.0	no
16	-1.669562022371790	-1.213816252342310	0.0	0.0	0.0	no
17	-1.4219387485277700	1.4687262193599300	0.0	0.0	0.0	yes
18	-0.183822379307677	0.7980906014343700	1.0	0.0	0.0	no
19	0.5590474422243790	-0.20786282545396800	0.0	0.0	0.0	no
20	0.31142416838036	-1.5491340613050800	0.0	0.0	0.0	no

	A	B	C	D	E	F
	train_preprocessed					
1	num_campaign	num_previous	cat_job_blue-collar	cat_month_jun	cat_month_sep	y
2	1.301917263756430	1.4687262193599300	0.0	0.0	0.0	no
3	1.5495405376004500	1.1334084103971500	0.0	0.0	0.0	no
4	-1.4219387485277700	-0.878498443379526	0.0	0.0	0.0	no
5	-1.4219387485277700	-1.213816252342310	0.0	0.0	1.0	no
6	1.0542939899124200	-1.5491340613050800	0.0	0.0	0.0	no
7	-0.9266922008397330	1.4687262193599300	0.0	0.0	0.0	no
8	-0.4314456531516960	-0.878498443379526	0.0	0.0	0.0	no
9	-0.679068926995714	0.4627727924715910	0.0	0.0	0.0	no
10	-0.679068926995714	-0.878498443379526	0.0	0.0	0.0	no
11	0.06380089453634150	1.1334084103971500	0.0	0.0	0.0	no
12	0.31142416838036	0.7980906014343700	0.0	0.0	1.0	no
13	-1.4219387485277700	0.4627727924715910	0.0	0.0	0.0	no
14	1.0542939899124200	1.4687262193599300	0.0	0.0	0.0	no
15	-1.174315474683750	-0.878498443379526	1.0	0.0	0.0	no
16	0.31142416838036	-0.5431806344167470	0.0	0.0	0.0	no
17	-0.9266922008397330	-0.20786282545396800	0.0	0.0	0.0	no
18	-1.174315474683750	-0.20786282545396800	0.0	0.0	0.0	no
19	0.31142416838036	1.4687262193599300	0.0	0.0	0.0	no
20	1.301917263756430	-0.5431806344167470	0.0	0.0	0.0	no

RESULT: Hence, various data preprocessing techniques were performed.

EXP NO: 3**WEB SCRAPING AND VISUALISATION****DATE:**

AIM: To scrape headlines from the given website and analyse the most frequent words.

QUESTIONS:

1. Fetch HTML content from <https://www.espncricinfo.com/> and extract all headlines present in <h2> and <h3> tags. Print the first 5 headlines.

CODE:

```

pip install requests beautifulsoup4

pip install curl-cffi beautifulsoup4

from bs4 import BeautifulSoup
from curl_cffi import requests

url = "https://www.espncricinfo.com/"

try:
    response = requests.get(url, impersonate="chrome", timeout=10)
    response.raise_for_status()

    soup = BeautifulSoup(response.text, "html.parser")

    headlines = []
    for tag in soup.find_all(["h2", "h3"]):
        text = tag.get_text(strip=True)

        if text and text not in headlines:
            headlines.append(text)

    print("--- Top 5 Headlines from Cricinfo ---")
    for i, headline in enumerate(headlines[:5], 1):
        print(f'{i}. {headline}')

except Exception as e:
    print(f'An error occurred while fetching the page: {e}')

```

OUTPUT:

```

--- Top 5 Headlines from Cricinfo ---
1. Match Coverage
2. Salt: 'Sometimes you've got to fight harder'
3. Gambhir: 'Middle order needs to play the situation'
4. Watch: India capitulate in powerplay
5. Watch – India all out for 76

```

2. From extracted headlines:

- **Convert text to lowercase**
- **Remove numbers, punctuations and special characters**
- **Remove common stop words**
- **Display first 20 cleaned words**

CODE:

```

import re
import string
from bs4 import BeautifulSoup
from curl_cffi import requests

STOPWORDS = {
    "the", "and", "to", "of", "in", "for", "on", "at", "with", "a", "and", "by", "is", "are", "this",
    "that"
}
all_text = ".join(headlines)
if all_text:
    lowercase_text = all_text.lower()

    clean_text = re.sub(r"[^a-z\s]", "", lowercase_text)

    words = clean_text.split()

    filtered_words = [word for word in words if word not in STOPWORDS]

    print("--- First 20 Cleaned Words ---")
    print(filtered_words[:20])

```

OUTPUT:

```

--- First 20 Cleaned Words ---
['match', 'coverage', 'salt', 'sometimes', 'youve', 'got', 'fight', 'harder', 'gambhir', 'middle', 'order',
'needs', 'play', 'situation', 'watch', 'india', 'capitulate', 'powerplay', 'watch', 'india']

```

```

--- First 20 Cleaned Words ---
['match', 'coverage', 'salt', 'sometimes', 'youve', 'got', 'fight', 'harder', 'gambhir', 'middle', 'order', 'needs', 'play', 'situation', 'watch',

```

3. Count the frequency of all words in cleaned text and display the Top 10 most frequent words along with their count.

CODE:

```
pip install matplotlib wordcloud

import collections
import re
from bs4 import BeautifulSoup
from curl_cffi import requests
import matplotlib.pyplot as plt
from wordcloud import WordCloud

word_counts = collections.Counter(filtered_words)
sorted_word_counts = word_counts.most_common()

print("--- Word Frequencies (Descending Order) ---")
for word, count in sorted_word_counts[:10]:
    print(f'{word}: {count}')
```

OUTPUT:

```
--- Word Frequencies (Descending Order) ---
watch: 11
out: 6
two: 4
india: 3
play: 2
all: 2
archer: 2
sooryavanshi: 2
ti: 2
pretorius: 2
```

4. Plot the frequency and words as a Bar Chart and a Word Cloud Image. Save it as an image.

CODE:

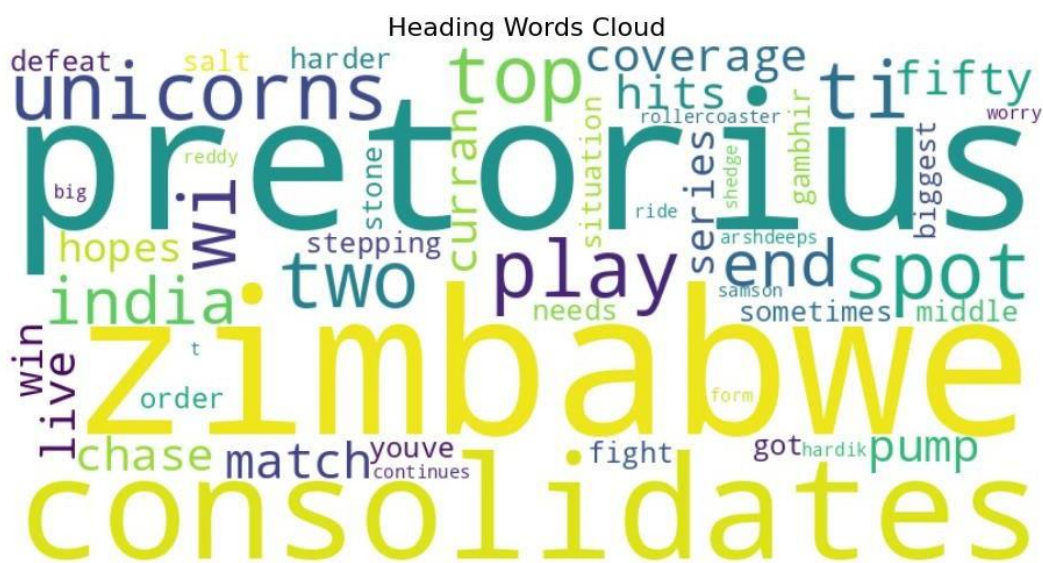
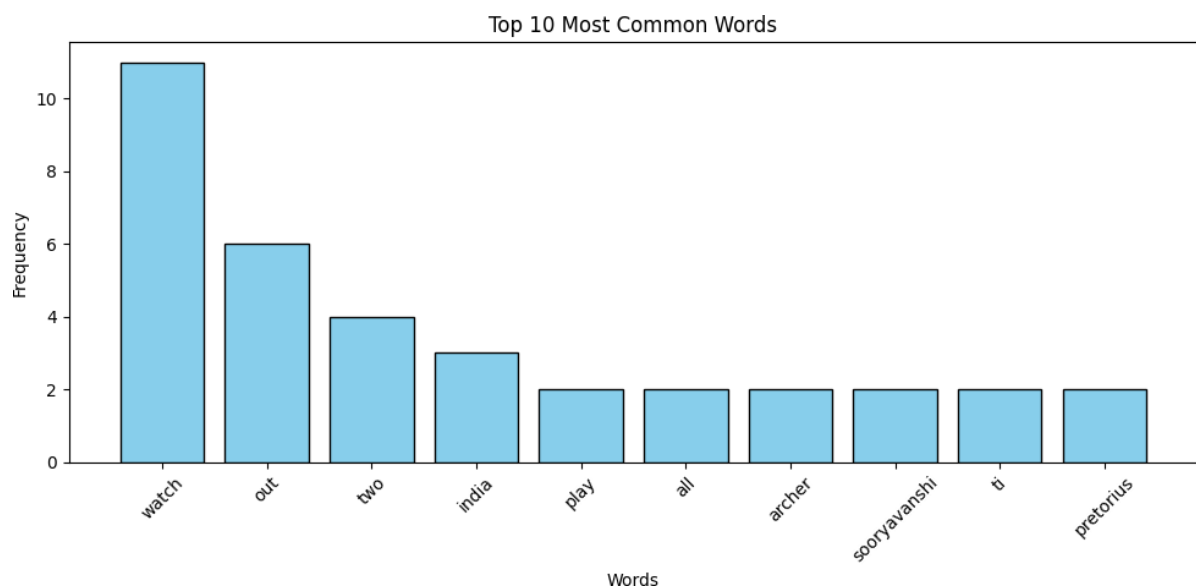
```
top_words = [item[0] for item in sorted_word_counts[:10]]
top_counts = [item[1] for item in sorted_word_counts[:10]]

plt.figure(figsize=(10, 5))
plt.bar(top_words, top_counts, color="skyblue", edgecolor="black")
plt.title("Top 10 Most Common Words")
plt.xlabel("Words")
plt.ylabel("Frequency")
```

```
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
text_for_cloud = " ".join(filtered_words)
wordcloud = WordCloud(width=800, height=400, background_color="white",
max_words=50).generate(text_for_cloud)
plt.savefig("word_cloud.png", dpi=300, bbox_inches='tight', facecolor='white')
plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")
plt.title("Heading Words Cloud", fontsize=16)
plt.tight_layout()
plt.show()
```

OUTPUT:



5. Visualise the Word Cloud Image using matplotlib.

CODE:

```
import random
if all_text:
    word_counts = collections.Counter(filtered_words)
    top_words = word_counts.most_common(40)
    max_count = top_words[0][1] if top_words else 1

    fig, ax = plt.subplots(figsize=(10, 6), facecolor="white")
    colors = [
        "#1f77b4",
        "#ff7f0e",
        "#2ca02c",
        "#d62728",
        "#9467bd",
        "#8c564b",
        "#e377c2",
        "#17becf",
    ]

    for word, count in top_words:
        font_size = 12 + int((count / max_count) * 36)
        x = random.uniform(0.1, 0.9)
        y = random.uniform(0.1, 0.9)

        ax.text(
            x,
            y,
            word,
            fontsize=font_size,
            color=random.choice(colors),
            ha="center",
            va="center",
            alpha=0.85,
            weight="bold",
        )

    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.axis("off")

    plt.title("Pure Matplotlib Word Cloud", fontsize=16, pad=20, weight="bold")
    plt.tight_layout()
    plt.savefig("word_cloud.png", dpi=300, bbox_inches="tight", facecolor="white")
    plt.show()
```

OUTPUT:

Pure Matplotlib Word Cloud



RESULT: Hence, headlines were scraped from the given website and the most frequent words were analysed.

EXP NO: 4
DATE: 16/7/26

DATA VISUALISATION IN DIFFERENT DIMENSIONS

AIM: To visualise data in different dimensions (1D, 2D, 3D and higher) using python libraries and to extract meaningful patterns and insights.

QUESTIONS:

I. Data Visualization Using the Iris Dataset

1.1 Plot a histogram for the sepal length feature of the Iris dataset. Analyse the shape of its distribution.

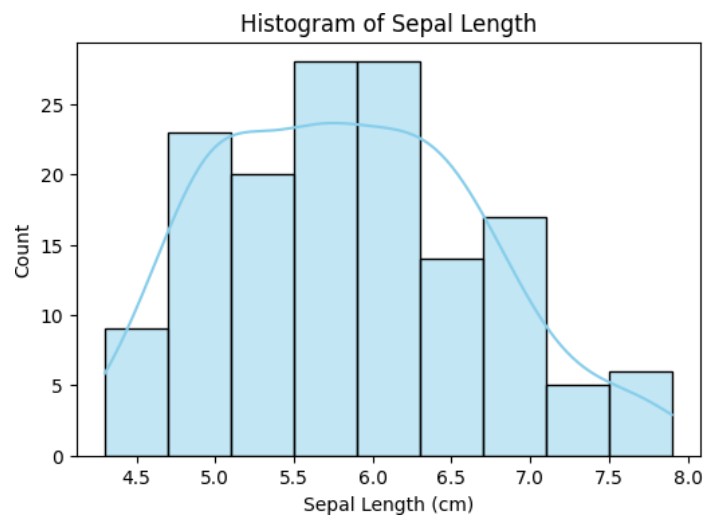
CODE:

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.datasets import load_iris

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)

plt.figure(figsize=(6, 4))
sns.histplot(
    data=df, x="sepal length (cm)", kde=True, color="skyblue"
)
plt.title("Histogram of Sepal Length")
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Count")
plt.show()
```

OUTPUT:



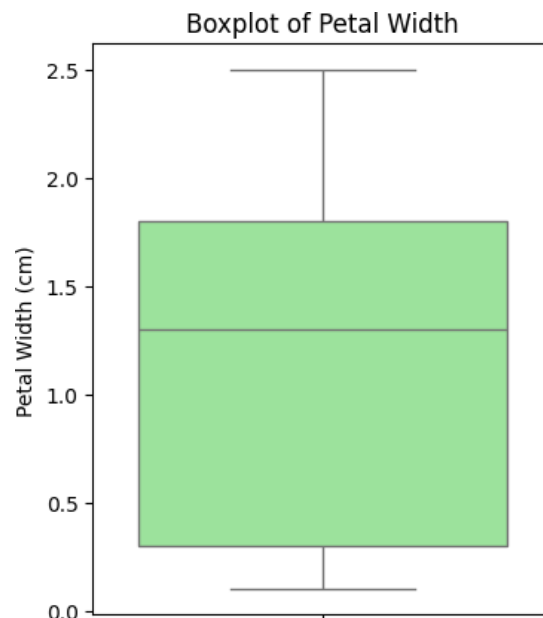
INFERENCE: The histogram shows that the sepal length is approximately normally distributed with a slight positive (right) skew. The distribution is nearly symmetric and bell-shaped, indicating that most values are concentrated around the mean, with a few observations extending toward higher sepal lengths.

1.2 Draw a box plot for the petal width feature of the Iris dataset. Identify any outliers, if present.

CODE:

```
plt.figure(figsize=(4, 5))
sns.boxplot(data=df, y="petal width (cm)", color="lightgreen")
plt.title("Boxplot of Petal Width")
plt.ylabel("Petal Width (cm)")
plt.show()
```

OUTPUT:



INFERENCE: The box plot indicates that there are no significant outliers in the petal width feature. All observations lie within the upper and lower whiskers, showing that the data is well distributed without any extreme values.

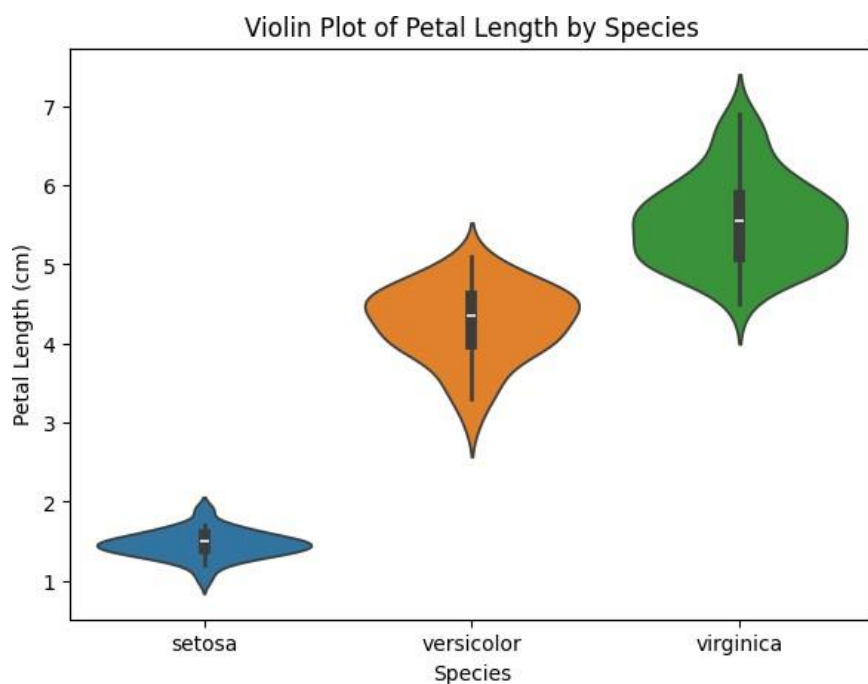
1.3 Create a violin plot for the petal length feature categorized by species. Interpret the observations from the plot.

CODE:

```
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
species_map = {0: "setosa", 1: "versicolor", 2: "virginica"}
df["species"] = [species_map[x] for x in iris.target]
```

```
plt.figure(figsize=(7, 5))
sns.violinplot(
    data=df,
    x="species",
    y="petal length (cm)",
    hue="species",
    legend=False,
)
plt.title("Violin Plot of Petal Length by Species")
plt.xlabel("Species")
plt.ylabel("Petal Length (cm)")
plt.show()
```

OUTPUT:



INFERENCE: The violin plot shows that Setosa has a distinct and compact distribution of petal lengths, making it clearly separable from the other two species. Versicolor and Virginica exhibit slightly overlapping distributions, indicating that petal length alone can effectively distinguish Setosa but may require additional features to accurately differentiate Versicolor and Virginica.

II. Two-Dimensional Visualization Using the Iris Dataset

1.4 Generate a scatter plot of sepal length versus sepal width, color-coded by species. What pattern can be observed from the plot?

CODE:

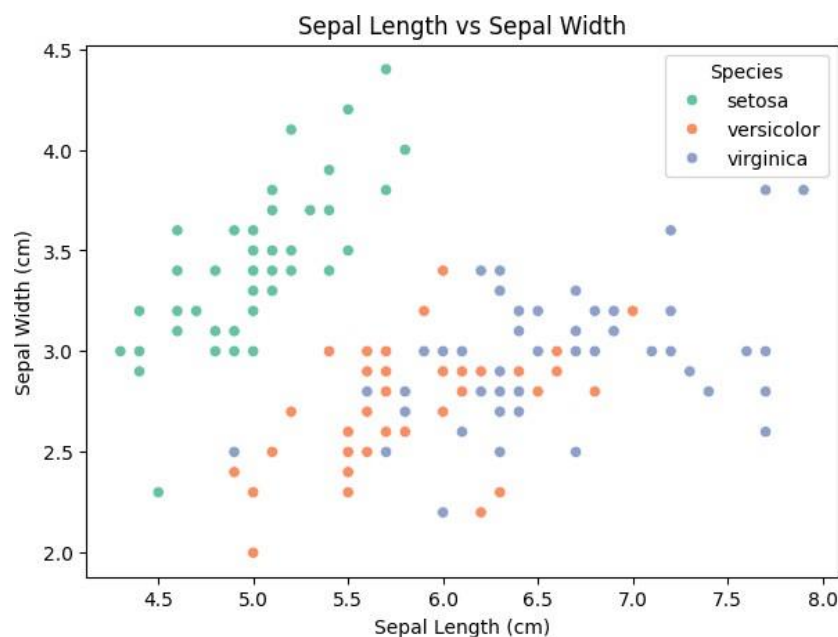
```
plt.figure(figsize=(7, 5))
sns.scatterplot(
```

```

data=df,
x="sepal length (cm)",
y="sepal width (cm)",
hue="species",
palette="Set2",
)
plt.title("Sepal Length vs Sepal Width")
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Sepal Width (cm)")
plt.legend(title="Species")
plt.show()

```

OUTPUT:



INFERENCE: The scatter plot shows that Setosa forms a distinct cluster with shorter sepal lengths and wider sepal widths. In contrast, Versicolor and Virginica exhibit considerable overlap, indicating that sepal measurements alone are not sufficient to clearly distinguish these two species.

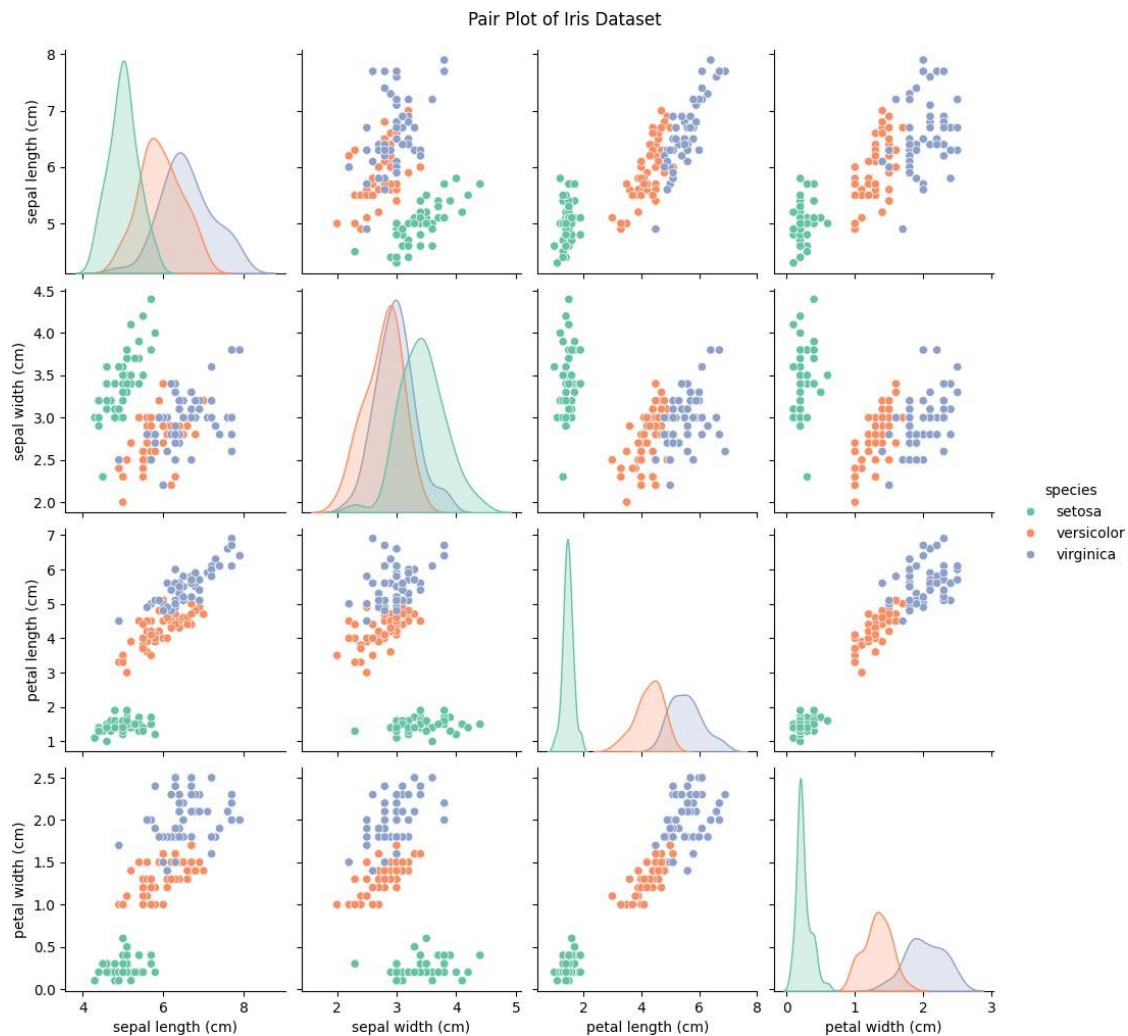
1.5 Create a pair plot (scatter matrix) for the Iris dataset. How does this visualization help in understanding the relationships among features?

CODE:

```

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
species_map = {0: "setosa", 1: "versicolor", 2: "virginica"}
df["species"] = [species_map[x] for x in iris.target]
sns.pairplot(data=df, hue="species", palette="Set2")
plt.suptitle("Pair Plot of Iris Dataset", y=1.02)
plt.show()

```

OUTPUT:

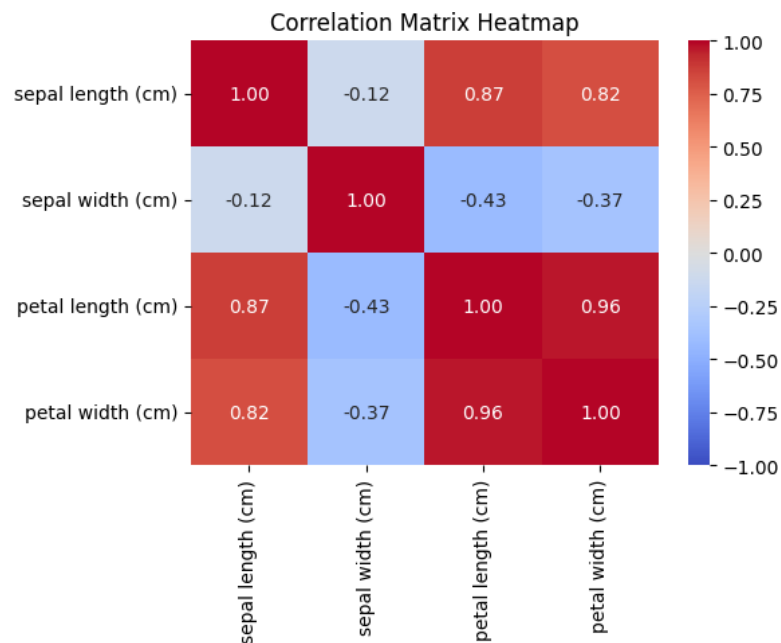
INFERENCE: The pair plot provides a comprehensive view of the relationships between all pairs of features. It helps identify feature distributions, correlations, and class separations, making it easier to determine which feature combinations are most effective for distinguishing the different Iris species.

1.6 Plot a heatmap of the correlation matrix for the Iris dataset. Which features are most strongly correlated?

CODE:

```
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
corr_matrix = df.corr()

plt.figure(figsize=(6, 5))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f", vmin=-1, vmax=1)
plt.title("Correlation Matrix Heatmap")
plt.tight_layout()
plt.show()
```

OUTPUT:

INFERENCE: The correlation heatmap shows that petal length and petal width have the strongest positive correlation. It also indicates that sepal length is positively correlated with both petal length and petal width, suggesting that petal-related features are the most informative for analysing and classifying the Iris dataset.

III. Three-Dimensional Visualization Using the Iris Dataset

1.7 Plot a three-dimensional scatter plot using sepal length, sepal width, and petal length. Use different colors to represent each Iris species.

CODE:

```
colors = {0: "teal", 1: "orange", 2: "purple"}
species_names = {0: "setosa", 1: "versicolor", 2: "virginica"}
df["color"] = [colors[x] for x in iris.target]
```

```
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")
```

```
scatter = ax.scatter(
    df["sepal length (cm)"],
    df["sepal width (cm)"],
    df["petal length (cm)"],
    c=df["color"],
    s=40,
    edgecolor="k",
)
```

```
ax.set_title("3D Scatter Plot of Iris Dataset")
```



```

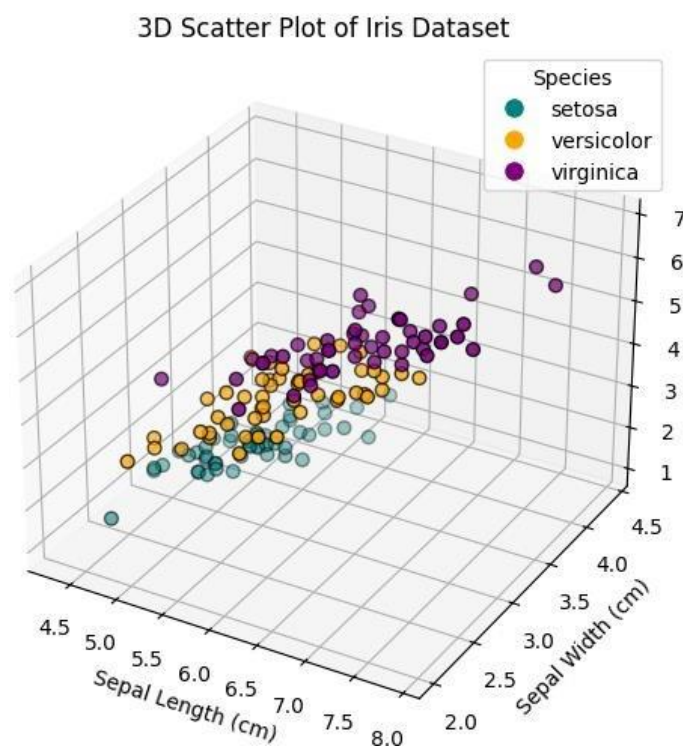
ax.set_xlabel("Sepal Length (cm)")
ax.set_ylabel("Sepal Width (cm)")
ax.set_zlabel("Petal Length (cm)")

markers = [
    plt.Line2D([0], [0], marker="o", color="w", markerfacecolor=col, markersize=10)
    for col in colors.values()
]
ax.legend(markers, species_names.values(), title="Species")

plt.show()

```

OUTPUT:



INFERENCE: The 3D scatter plot provides a clearer visualization of the relationships among the three features. Setosa forms a well-separated cluster, while Versicolor and Virginica are closer together with slight overlap. The use of three dimensions enhances the visualization of feature variations and improves the understanding of species-wise data distribution.

IV. Data Visualization Using the Titanic Dataset

2.1 Plot a count plot for the *survived* column. Analyse the distribution of passenger survival.

CODE:

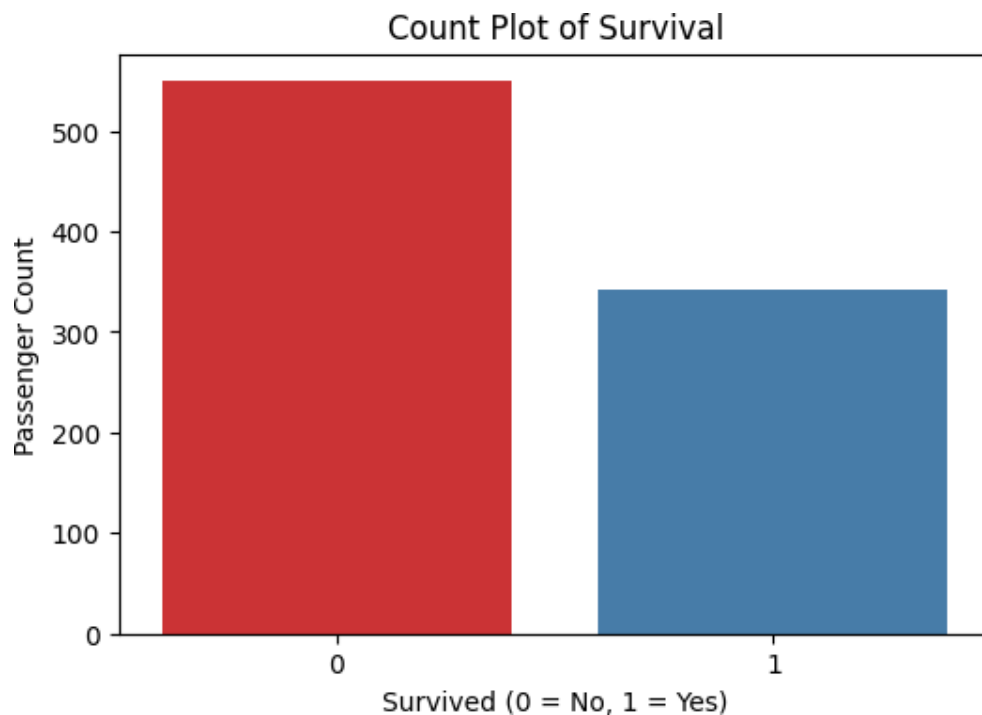
```

import matplotlib.pyplot as plt
import seaborn as sns

```

```
titanic = sns.load_dataset("titanic")

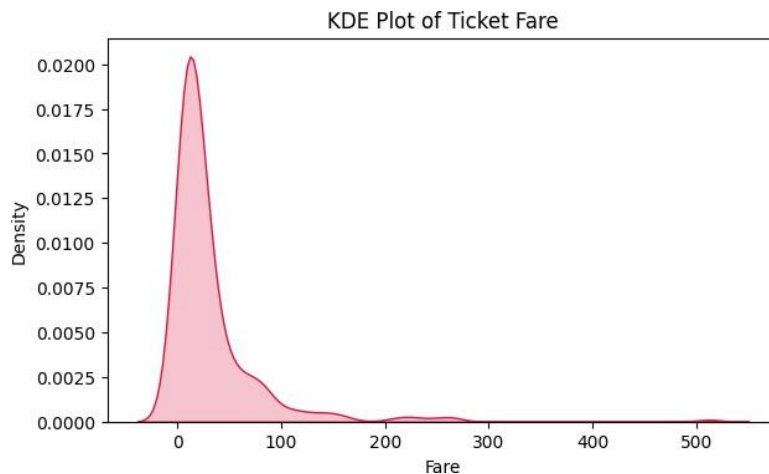
plt.figure(figsize=(6, 4))
sns.countplot(data=titanic, x="survived", hue="survived", palette="Set1", legend=False)
plt.title("Count Plot of Survival")
plt.xlabel("Survived (0 = No, 1 = Yes)")
plt.ylabel("Passenger Count")
plt.show()
```

OUTPUT:

INFERENCE: The count plot shows that the survival distribution is imbalanced, with a greater number of passengers not surviving than surviving. Approximately 61% of the passengers perished, while only about 39% survived, indicating that fatalities were more common in the Titanic disaster.

2.2 Visualize the distribution of ticket fares using a Kernel Density Estimation (KDE) plot. Is the distribution skewed?**CODE:**

```
plt.figure(figsize=(7, 4))
sns.kdeplot(data=titanic, x="fare", fill=True, color="crimson")
plt.title("KDE Plot of Ticket Fare")
plt.xlabel("Fare")
plt.ylabel("Density")
plt.show()
```

OUTPUT:

INFERENCE: The KDE plot reveals that the fare distribution is positively (right) skewed. Most passengers paid relatively low fares, while a small number of passengers purchased very expensive tickets, resulting in a long tail toward higher fare values.

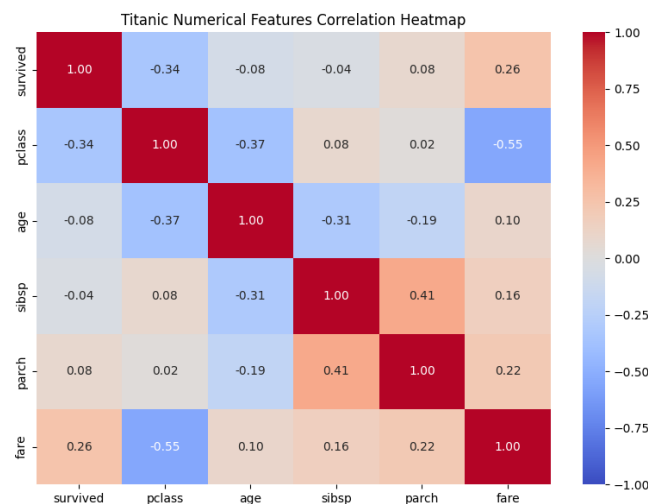
2.3 Display a correlation heatmap for the numerical features of the Titanic dataset. Which variables show the strongest correlations?

CODE:

```
numeric_df = titanic.select_dtypes(include=["float64", "int64"])

corr_matrix = numeric_df.corr()

plt.figure(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f", vmin=-1, vmax=1)
plt.title("Titanic Numerical Features Correlation Heatmap")
plt.tight_layout()
plt.show()
```

OUTPUT:

INFERENCE: The heatmap indicates that passenger class (Pclass) and fare have a strong negative correlation, showing that higher-class passengers generally paid higher fares. It also reveals a moderate positive correlation between SibSp and Parch, suggesting that family members often travelled together. Additionally, Pclass and Age exhibit a mild negative correlation.

2.4 Create a three-dimensional scatter plot using *fare*, *age*, and *Pclass*, with data points color-coded by the *survived* column. Interpret the observed pattern.

CODE:

```
titanic = sns.load_dataset("titanic").dropna(subset=["age", "fare", "pclass"])

colors = {0: "crimson", 1: "teal"}
passenger_colors = [colors[x] for x in titanic["survived"]]

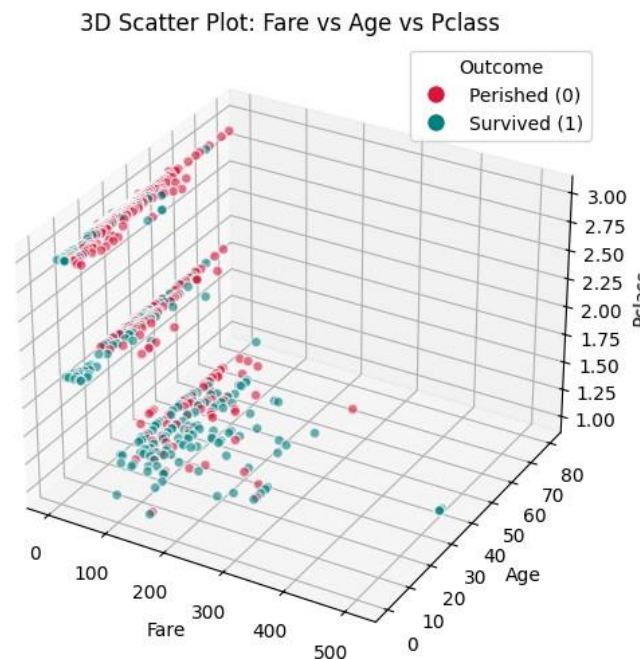
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")

ax.scatter(
    titanic["fare"],
    titanic["age"],
    titanic["pclass"],
    c=passenger_colors,
    alpha=0.6,
    edgecolor="w",
    s=30
)

ax.set_title("3D Scatter Plot: Fare vs Age vs Pclass")
ax.set_xlabel("Fare")
ax.set_ylabel("Age")
ax.set_zlabel("Pclass")

markers = [
    plt.Line2D([0], [0], marker="o", color="w", markerfacecolor=col, markersize=10)
    for col in colors.values()
]
ax.legend(markers, ["Perished (0)", "Survived (1)"], title="Outcome")

plt.show()
```

OUTPUT:

INFERENCE: The 3D scatter plot shows that passengers in first class with higher ticket fares had a greater likelihood of survival. In contrast, most third-class passengers with lower fares are clustered among the non-survivors. The visualization also indicates that younger passengers had relatively better survival rates, reflecting the historical rescue priority given to women and children.

2.5 Apply Principal Component Analysis (PCA) to reduce the dataset dimensions and visualize the transformed data using a three-dimensional scatter plot.

CODE:

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

titanic = sns.load_dataset("titanic")
features = ["pclass", "age", "sibsp", "parch", "fare"]

df_pca = titanic[features + ["survived"]].dropna()

X = df_pca[features]
y = df_pca["survived"]

X_scaled = StandardScaler().fit_transform(X)

pca = PCA(n_components=3)
X_pca = pca.fit_transform(X_scaled)

pca_df = pd.DataFrame(data=X_pca, columns=["PC1", "PC2", "PC3"])
pca_df["survived"] = y.values
```

```

colors = {0: "crimson", 1: "teal"}
dot_colors = [colors[x] for x in pca_df["survived"]]

fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")

ax.scatter(
    pca_df["PC1"],
    pca_df["PC2"],
    pca_df["PC3"],
    c=dot_colors,
    alpha=0.6,
    edgecolor="w",
    s=30
)

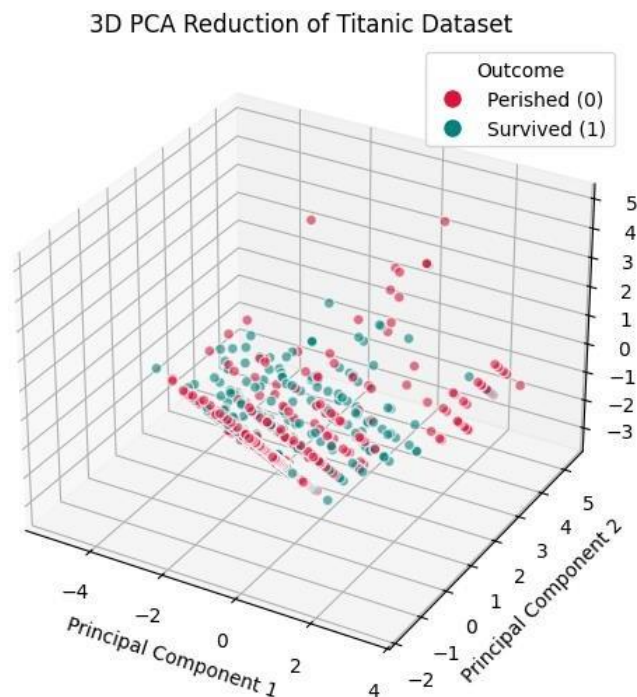
ax.set_title("3D PCA Reduction of Titanic Dataset")
ax.set_xlabel("Principal Component 1")
ax.set_ylabel("Principal Component 2")
ax.set_zlabel("Principal Component 3")

markers = [
    plt.Line2D([0], [0], marker="o", color="w", markerfacecolor=col, markersize=10)
    for col in colors.values()
]
ax.legend(markers, ["Perished (0)", "Survived (1)"], title="Outcome")

plt.show()

```

OUTPUT:



INFERENCE: The PCA plot successfully reduces the original numerical features into three principal components while preserving most of the dataset's variance. Although the dimensionality is reduced, the survivor and non-survivor classes still overlap considerably, indicating that the data is not linearly separable and may require more advanced machine learning models for accurate classification.

V. Multidimensional Visualization Using the Wine Dataset

3.1 Create a scatter plot of *Alcohol* versus *Colour Intensity*, using different colours to represent the wine classes. Interpret the observed pattern.

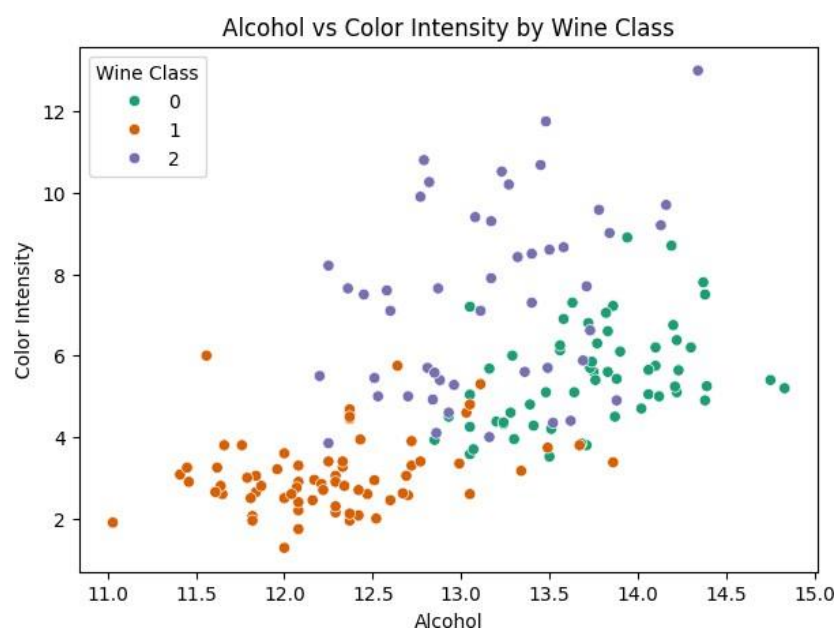
CODE:

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.datasets import load_wine

wine = load_wine()
df = pd.DataFrame(wine.data, columns=wine.feature_names)
df["class"] = wine.target

plt.figure(figsize=(7, 5))
sns.scatterplot(
    data=df, x="alcohol", y="color_intensity", hue="class", palette="Dark2"
)
plt.title("Alcohol vs Color Intensity by Wine Class")
plt.xlabel("Alcohol")
plt.ylabel("Color Intensity")
plt.legend(title="Wine Class")
plt.show()
```

OUTPUT:



INFERENCE: The scatter plot shows that the wine classes form distinct clusters based on alcohol content and colour intensity. Class 0 generally exhibits higher alcohol content, while the remaining classes occupy different regions of the plot. This indicates that these features are effective in distinguishing between the wine classes.

3.2 Create a parallel coordinates plot for all features, with lines color-coded according to the wine class. What observations can be made from the visualization?

CODE:

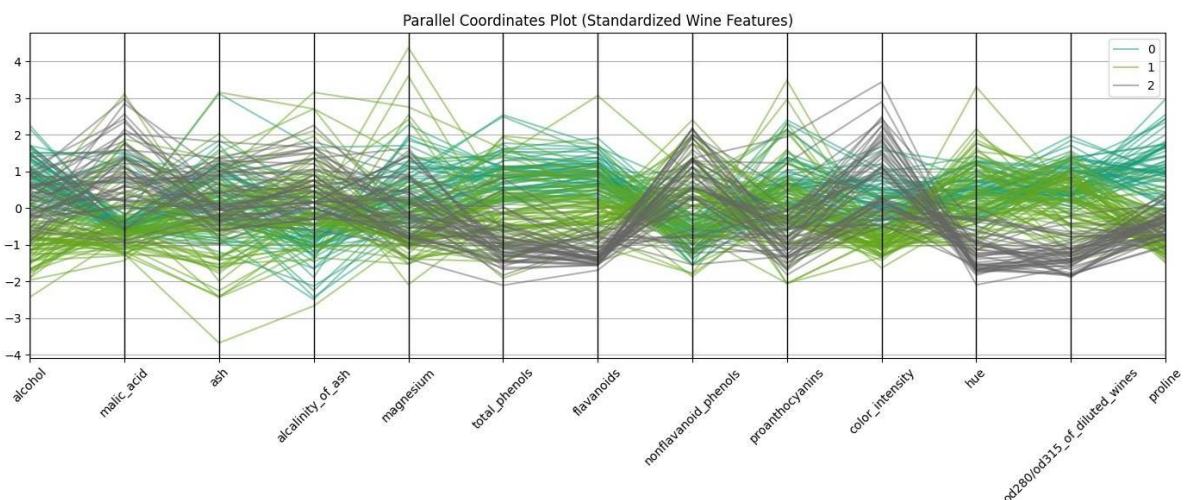
```
from pandas.plotting import parallel_coordinates
from sklearn.preprocessing import StandardScaler

wine = load_wine()
df = pd.DataFrame(wine.data, columns=wine.feature_names)

scaler = StandardScaler()
scaled_features = scaler.fit_transform(df)
df_scaled = pd.DataFrame(scaled_features, columns=wine.feature_names)
df_scaled["class"] = wine.target

plt.figure(figsize=(14, 6))
parallel_coordinates(df_scaled, "class", colormap="Dark2", alpha=0.5)
plt.title("Parallel Coordinates Plot (Standardized Wine Features)")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

OUTPUT:



INFERENCE: The parallel coordinates plot reveals that samples belonging to the same wine class follow similar patterns across multiple features. Certain attributes, such as Proline, Flavanoids, and Alcohol, clearly separate the classes, while features like Ash and Magnesium show greater overlap. This visualization highlights the multidimensional relationships among

the features and demonstrates that several chemical properties effectively distinguish the wine classes.

3.3 Plot a radar chart showing the average feature values for each wine class. What observations can be made from the chart?

CODE:

```
import numpy as np
from sklearn.preprocessing import MinMaxScaler

wine = load_wine()
df = pd.DataFrame(wine.data, columns=wine.feature_names)
df["class"] = wine.target

scaler = MinMaxScaler()
df_scaled = pd.DataFrame(scaler.fit_transform(df[wine.feature_names]),
                          columns=wine.feature_names)
df_scaled["class"] = df["class"]

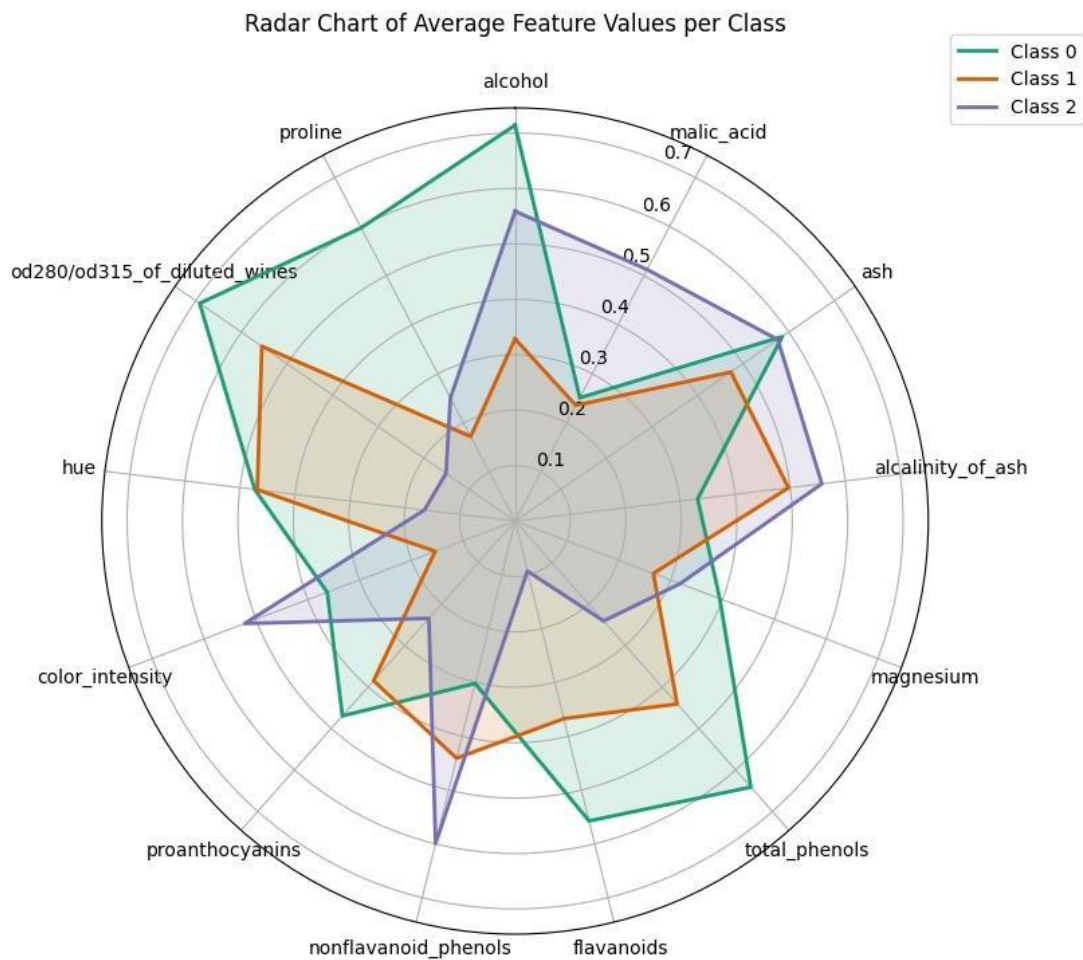
class_means = df_scaled.groupby("class").mean()
features = list(class_means.columns)
num_vars = len(features)

angles = np.linspace(0, 2 * np.pi, num_vars, endpoint=False).tolist()
angles += angles[:1]

fig, ax = plt.subplots(figsize=(8, 8), subplot_kw=dict(polar=True))
colors = ["#1b9e77", "#d95f02", "#7570b3"]

for i, class_label in enumerate(class_means.index):
    values = class_means.loc[class_label].tolist()
    values += values[:1]
    ax.plot(angles, values, color=colors[i], linewidth=2, label=f"Class {class_label}")
    ax.fill(angles, values, color=colors[i], alpha=0.15)

ax.set_theta_offset(np.pi / 2)
ax.set_theta_direction(-1)
ax.set_thetagrids(np.degrees(angles[:-1]), features)
plt.title("Radar Chart of Average Feature Values per Class", y=1.08)
plt.legend(loc="upper right", bbox_to_anchor=(1.2, 1.1))
plt.show()
```

OUTPUT:

INFERENCE: The radar chart illustrates that each wine class has a unique chemical profile. Class 0 is characterized by higher values of Alcohol and Proline, while Class 2 exhibits higher Malic Acid and Alkalinity of Ash. The distinct shapes of the radar plots indicate that the wine classes possess different characteristics, making them suitable for accurate classification.

VI. Map-Based Visualization Using the COVID-19 Dataset

4.1 Using the dataset containing COVID-19 cases and ISO country codes, create a choropleth map to visualize the number of confirmed COVID-19 cases across the world.

CODE:

```

pip install plotly
pip install pycountry
import pandas as pd
import plotly.express as px
import plotly.io as pio
import pycountry

pio.renderers.default = "iframe"

```

```

def get_iso3(country_name):
    try:
        return pycountry.countries.lookup(country_name).alpha_3
    except LookupError:
        mapping = {
            "US": "USA",
            "USA": "USA",
            "United States": "USA",
            "UK": "GBR",
            "United Kingdom": "GBR",
            "South Korea": "KOR",
            "Korea, South": "KOR",
            "Taiwan*": "TWN",
            "Congo (Kinshasa)": "COD",
            "Congo (Brazzaville)": "COG",
        }
        return mapping.get(country_name, None)

df = pd.read_csv("covid dataset.csv")
df["iso_alpha"] = df["Country/Region"].apply(get_iso3)

fig = px.choropleth(
    data_frame=df,
    locations="iso_alpha",
    locationmode="ISO-3",
    color="Confirmed",
    hover_name="Country/Region",
    hover_data=["Deaths", "Recovered", "Active"],
    color_continuous_scale=px.colors.sequential.Plasma,
    title="Global Distribution of Confirmed COVID-19 Cases",
)

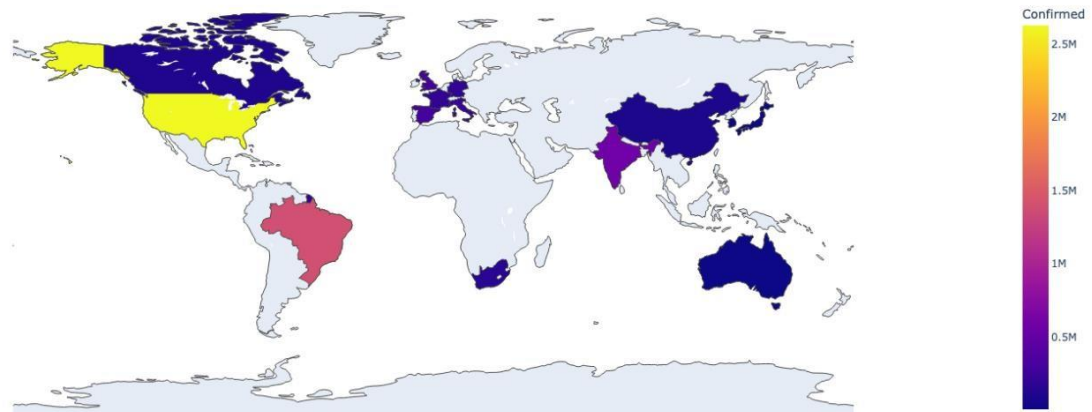
fig.update_layout(
    geo=dict(showframe=False, showcoastlines=True, projection_type="equiangular"),
    margin={"r": 0, "t": 40, "l": 0, "b": 0},
)

fig.show()

```

OUTPUT:

Global Distribution of Confirmed COVID-19 Cases



INFERENCE: The choropleth map clearly illustrates the global distribution of confirmed COVID-19 cases. Countries with a higher number of confirmed cases appear in darker shades, making it easy to identify the regions most affected by the pandemic. The visualization provides an effective overview of the geographical spread and severity of COVID-19 worldwide.

4.2 Build an interactive map using Plotly to visualize the confirmed, recovered, and death counts for each country. Which regions show higher mortality rates?

CODE:

```
import pandas as pd
import plotly.express as px
import plotly.io as pio
import pycountry

pio.renderers.default = "iframe"

def get_iso3(country_name):
    try:
        return pycountry.countries.lookup(country_name).alpha_3
    except LookupError:
        mapping = {
            "US": "USA",
            "USA": "USA",
            "United States": "USA",
            "UK": "GBR",
            "United Kingdom": "GBR",
            "South Korea": "KOR",
            "Korea, South": "KOR",
            "Taiwan*": "TWN",
            "Congo (Kinshasa)": "COD",
            "Congo (Brazzaville)": "COG",
        }
        return mapping.get(country_name, None)
```

```

df = pd.read_csv("covid dataset.csv")
df["iso_alpha"] = df["Country/Region"].apply(get_iso3)

fig = px.scatter_geo(
    data_frame=df,
    locations="iso_alpha",
    locationmode="ISO-3",
    size="Confirmed",
    color="Deaths / 100 Cases",
    hover_name="Country/Region",
    hover_data=[
        "Confirmed",
        "Deaths",
        "Recovered",
        "Deaths / 100 Cases",
    ],
    color_continuous_scale=px.colors.sequential.Reds,
    title="Interactive COVID-19 Map: Size (Confirmed) vs. Color (Mortality Rate)",
)

fig.update_layout(margin={"r": 0, "t": 40, "l": 0, "b": 0})

fig.show()

```

OUTPUT:

Interactive COVID-19 Map: Size (Confirmed) vs. Color (Mortality Rate)



INFERENCE: The interactive map enables easy comparison of COVID-19 statistics across countries by displaying confirmed cases, recoveries, and mortality rates. Countries with higher mortality rates are represented using darker shades, while larger markers indicate a greater number of confirmed cases. The visualization helps identify regions that experienced relatively severe outcomes and supports comparative analysis of the pandemic's global impact.

RESULT: Hence, data was visualised in different dimensions and meaningful patterns and insights were extracted successfully.